ANNEXE A BIS — SCHÉMA RELATIONNEL CORRIGÉ

Modèle de données CoGes V1.1

Version corrective du modèle V1. Les quatre décisions ouvertes sont verrouillées, dix défauts structurels identifiés sont corrigés, trois tables sont ajoutées et six sont amendées. Le périmètre fonctionnel de CoGes est inchangé : aucune extension future n'est implémentée, seules les structures nécessaires à la non-régression de l'évolution sont préparées. Toute décision V1 pertinente est conservée telle quelle.

TABLES AJOUTÉES
3
forecasts · insights · sale_return_refunds

TABLES AMENDÉES
6
products · sales · sale_items · inventory_movements · product_stock · daily_sales_facts

DÉCISIONS VERROUILLÉES
4
variantes · compte unique · stock ≥ 0 · CMP

SOMMAIRE

1. Résumé des corrections
2. Décisions définitivement verrouillées
3. Diagramme ER complet V1.1
4. Dictionnaire de données V1.1
5. Règles métier critiques
6. Architecture analytique
7. Isolation multi-tenant
8. SQL PostgreSQL 16 complet
9. Migration Alembic V1 → V1.1
10. Tests d'intégrité indispensables
11. Matrice V1 / V1.1 / futur
12. Critère final

1. Résumé des corrections

Problème V1	Correction V1.1	Impact
Aucune structure pour persister les résultats ML : l'API devrait exécuter un modèle pour répondre	Ajout de forecasts et insights, tenant-scoped, RLS, écrites par le pipeline, lues seules par l'API	Chemin de lecture utilisateur découplé du calcul ML. Aucune inférence en ligne.
Annulation de vente : les mouvements inverses ne sont pas rattachés au mouvement d'origine	inventory_movements.reversal_of_movement_id (FK composite auto-référente) + UNIQUE (company_id, reversal_of_movement_id)	Traçabilité exacte « quel mouvement annule quoi » et idempotence structurelle de l'annulation.
Remboursement d'un retour non traçable financièrement ; notion d'« avoir » ambiguë	payments.sale_return_id + table de lettrage sale_return_refunds + CHECK de direction + trigger de plafond	Lien retour ↔ trésorerie bidirectionnel. Double remboursement impossible. Avoir client déclaré **hors V1**.
Taxes ni figées ni exclues : une modification de règle pouvait réécrire une vente passée	Snapshot sale_items.tax_rate / tax_amount, sales.tax_total, sale_return_items.tax_refund ; products.default_tax_rate comme seule source de paramétrage	Montants historiques immuables. Égalité comptable sous-total − remises + taxes = total garantie en base.
CMP annoncé mais absent du modèle : aucune colonne ne porte le coût moyen	product_stock.avg_cost + stock_value généré, maintenus par le trigger de mouvement ; unit_cost des sorties estampillé au CMP applicable	Valeur de stock, coût de sortie et COGS historique calculables. Méthode explicite et testable.
daily_sales_facts multipliait les lignes quand un sale_item a plusieurs lignes de retour	Pré-agrégation des retours par sale_item_id dans une CTE avant jointure ; métriques redéfinies et renommées	Fin de la surestimation de quantité, CA, remises, COGS et nombre de ventes.
daily_sales_facts était une vue matérialisée : pas de RLS, fuite possible entre tenants	Transformation en **table d'agrégats** avec company_id, PK tenant-aware, RLS + FORCE RLS, reconstruite par le processus analytique	Isolation analytique au même niveau que le transactionnel.
audit_logs hors stratégie RLS alors qu'il contient des données d'entreprise	RLS + FORCE RLS sur audit_logs et sur toutes les nouvelles tables tenant-scoped	Plus aucune table tenant-scoped sans politique.
Risque de FK simples sur les nouvelles relations	Toutes les FK V1.1 sont composites (company_id, child_id) → (company_id, parent_id)	Référence croisée entre entreprises structurellement impossible.
Trois outils de migration cités en parallèle	Standardisation sur **Alembic + SQLAlchemy**, migration versionnée V1 → V1.1 non destructive	Une seule chaîne de migration, reproductible en CI.
Stock négatif « paramétrable par entreprise »	Interdit sans exception : CHECK sur qty_on_hand + rejet dans le trigger, pour tout type de sortie	Une seule sémantique de stock dans tout le système.
ml_feature_snapshots marquée « pas de la V1 » puis « V1.1 »	Statut tranché : **créée en V1.1**, alimentée par le pipeline, non lue par l'application	Fin de l'ambiguïté de périmètre.

Aucune table V1 n'est supprimée, aucune colonne n'est renommée hors couche analytique, aucun type énuméré n'est réduit. Les seuls renommages portent sur `daily_sales_facts`, qui est reconstruite intégralement et dont aucune donnée n'est source de vérité.

2. Décisions définitivement verrouillées

2.1 Variantes produit — pas de moteur de variantes en V1

`products` reste l'unité de référence unique des ventes et du stock. Il n'existe ni `product_variants`, ni matrice taille/couleur, ni SKU parent/enfant. Aucune notion de variante n'est codée en dur : ni colonne `size`, ni `color`, ni convention de nommage de SKU interprétée par le code.

Point d'extension documenté. Les quatre tables qui référencent un article le font par `product_id` et par lui seul : `sale_items`, `inventory_movements`, `product_stock`, `product_prices`. L'extension future consistera à créer `product_variants (id, company_id, product_id, attributes jsonb, sku)` puis à ajouter une colonne `variant_id` nullable à ces quatre tables, le NULL signifiant « l'article lui-même ». Cette évolution reste possible sans réécriture de l'historique parce que : (a) le stock est événementiel, donc une reventilation par variante n'exige aucun UPDATE des mouvements passés ; (b) `product_stock` est une projection jetable, sa clé peut devenir `(product_id, variant_id)` par reconstruction ; (c) le prix est historisé par intervalles, un prix par variante s'ajoute sans invalider les intervalles existants.

Non implémenté en V1.1. Aucune colonne `variant_id` n'est créée maintenant.

2.2 Utilisateur / entreprise — 1 compte = 1 entreprise

`users.company_id NOT NULL` est conservé comme unique rattachement. Il n'existe aucune table de membership. Un humain travaillant pour deux entreprises possède deux comptes distincts, avec deux mots de passe et deux sessions : `users.email` est unique **par entreprise** (`UNIQUE (company_id, email)`) et non globalement, ce qui rend ce cas représentable sans conflit.

Conséquence assumée sur l'authentification : l'identifiant de connexion n'est pas l'e-mail seul mais le couple (slug d'entreprise, e-mail). C'est ce qui permet au JWT de porter un `company_id` unique et non ambigu, donc au serveur d'émettre un `SET LOCAL app.company_id` déterministe à chaque transaction. Le multi-company introduirait un choix de tenant après authentification, et donc un risque de bascule de contexte — écarté en V1.

Extension future documentée : table `company_memberships (user_id, company_id, role_set)`, `users.company_id` devenant le tenant d'origine puis étant retiré après reprise des données.
Non implémenté.

2.3 Stock négatif — interdit par défaut, sans dérogation

Toute opération qui amènerait `qty_on_hand` sous zéro est rejetée par PostgreSQL, atomiquement, dans la transaction appelante. La règle s'applique indistinctement à la vente (`sale_out`), à l'ajustement négatif (`adjustment`), à la perte (`loss`), au retour fournisseur (`supplier_return_out`) et à toute sortie future. Elle est portée par deux mécanismes redondants : un `CHECK (qty_on_hand >= 0)` sur la projection et un `RAISE EXCEPTION` explicite dans le trigger de mouvement, qui produit un message métier exploitable par l'API.

La colonne de paramétrage par entreprise évoquée en V1 est **supprimée de la conception** : un stock négatif autorisé chez un tenant rendrait le CMP indéfini (division par une quantité nulle ou négative) et la valeur de stock non interprétable. Les articles `is_stock_tracked = false` restent hors du mécanisme : aucun mouvement ne peut être écrit pour eux, et aucune ligne de `product_stock` n'existe.

2.4 Valorisation — coût moyen pondéré (CMP)

Le CMP est la méthode unique de V1.1. FIFO n'est pas implémenté. Le modèle porte le CMP courant dans `product_stock.avg_cost`, recalculé à chaque entrée, et fige le coût de chaque sortie dans `inventory_movements.unit_cost`. `sale_items.unit_cost` reste le snapshot du coût appliqué à la vente.

Évolution future préservée : chaque mouvement conserve sa quantité signée, son coût unitaire et son horodatage métier. Un recalcul complet en FIFO ou en coût standard est donc possible *a posteriori* à partir du seul journal `inventory_movements`, sans donnée manquante. Seule la projection `product_stock` serait à reconstruire.

3. Diagramme ER complet V1.1

30 tables. Notation Mermaid : `||` exactement un, `o|` zéro ou un, `o{` zéro ou plusieurs, `|{` un ou plusieurs. Les relations ajoutées en V1.1 sont commentées `%% V1.1`. Toutes les relations traversant des données tenant-scoped sont matérialisées par des FK composites `(company_id, child_id) → (company_id, parent_id)` ; le diagramme ne montre que la colonne fonctionnelle.

```
erDiagram
    COMPANIES ||--o{ USERS : "emploie"
    COMPANIES ||--o{ ROLES : "definit"
    COMPANIES ||--o{ CUSTOMERS : "possede"
    COMPANIES ||--o{ SUPPLIERS : "possede"
    COMPANIES ||--o{ CATEGORIES : "possede"
    COMPANIES ||--o{ PRODUCTS : "possede"
    COMPANIES ||--o{ SALES : "enregistre"
    COMPANIES ||--o{ EXPENSES : "supporte"
    COMPANIES ||--o{ DEBTS : "suit"
    COMPANIES ||--o{ PAYMENTS : "encaisse"
    COMPANIES ||--o{ INVENTORY_MOVEMENTS : "tracte"
    COMPANIES ||--o{ AUDIT_LOGS : "journalise"
    COMPANIES ||--o{ BUSINESS_EVENTS : "publie"
    COMPANIES ||--o{ NOTIFICATIONS : "notifie"
    COMPANIES ||--o{ DAILY_SALES_FACTS : "agrege"
    COMPANIES ||--o{ ML_FEATURE_SNAPSHOTS : "featurise"
    COMPANIES ||--o{ FORECASTS : "prevoit"
    COMPANIES ||--o{ INSIGHTS : "observe"

    USERS ||--o{ USER_ROLES : "detient"
    ROLES ||--o{ USER_ROLES : "attribue_a"
    ROLES ||--o{ ROLE_PERMISSIONS : "accorde"
    PERMISSIONS ||--o{ ROLE_PERMISSIONS : "accordee_par"

    CATEGORIES ||--o{ CATEGORIES : "parent_de"
    CATEGORIES ||--o{ PRODUCTS : "classe"
    PRODUCTS ||--o{ PRODUCT_PRICES : "tarife_par"
    PRODUCTS ||--o| PRODUCT_STOCK : "etat_courant_cmp"
    PRODUCTS ||--o{ SALE_ITEMS : "vendu_dans"
    PRODUCTS ||--o{ INVENTORY_MOVEMENTS : "mouvemente"

    CUSTOMERS ||--o{ SALES : "achete"
    USERS ||--o{ SALES : "saisit"
    SALES ||--|{ SALE_ITEMS : "compose_de"
    SALES ||--o{ SALE_RETURNS : "retourne"
    SALE_RETURNS ||--|{ SALE_RETURN_ITEMS : "compose_de"
    SALE_ITEMS ||--o{ SALE_RETURN_ITEMS : "retourne_via"
    PRODUCT_PRICES |o--o{ SALE_ITEMS : "prix_applique"

    SALE_ITEMS ||--o| INVENTORY_MOVEMENTS : "sortie_stock"
    SALE_RETURN_ITEMS ||--o| INVENTORY_MOVEMENTS : "entree_retour"
    SUPPLIERS ||--o{ INVENTORY_MOVEMENTS : "approvisionne"
    INVENTORY_MOVEMENTS ||--o| INVENTORY_MOVEMENTS : "annule_par"          %% V1.1

    EXPENSE_CATEGORIES ||--o{ EXPENSES : "classe"
    SUPPLIERS ||--o{ EXPENSES : "facture"

    SALES ||--o{ DEBTS : "genere"
    CUSTOMERS ||--o{ DEBTS : "doit"
    SUPPLIERS ||--o{ DEBTS : "creancier"
    DEBTS ||--o{ PAYMENT_ALLOCATIONS : "reglee_par"
    PAYMENTS ||--o{ PAYMENT_ALLOCATIONS : "affecte_a"
```

```
    SALES ||--o{ PAYMENTS : "encaissement"
    EXPENSES ||--o{ PAYMENTS : "decaissement"
    USERS ||--o{ PAYMENTS : "saisit"
    SALE_RETURNS ||--o{ SALE_RETURN_REFUNDS : "rembourse_par"              %% V1.1
    PAYMENTS ||--o{ SALE_RETURN_REFUNDS : "couvre_retour"                  %% V1.1

    USERS ||--o{ AUDIT_LOGS : "acteur"
    USERS ||--o{ NOTIFICATIONS : "destinataire"
    PRODUCTS ||--o{ DAILY_SALES_FACTS : "agrege_par"
    PRODUCTS ||--o{ ML_FEATURE_SNAPSHOTS : "featurise"
    PRODUCTS ||--o{ FORECASTS : "prevu_pour"                               %% V1.1

    COMPANIES {
        uuid id PK
        text name
        citext slug UK
        char currency
        text timezone
        jsonb settings
        timestamptz deleted_at
    }
    USERS {
        uuid id PK
        uuid company_id FK
        citext email UK
        text password_hash
        text full_name
        boolean is_active
        timestamptz last_login_at
        timestamptz deleted_at
    }
    ROLES {
        uuid id PK
        uuid company_id FK
        text name UK
        boolean is_system
    }
    PERMISSIONS {
        uuid id PK
        text code UK
        text resource
        text action
    }
    USER_ROLES {
        uuid user_id PK
        uuid role_id PK
        uuid company_id FK
    }
    ROLE_PERMISSIONS {
        uuid role_id PK
        uuid permission_id PK
        uuid company_id FK
    }
    CUSTOMERS {
        uuid id PK
        uuid company_id FK
        text code UK
        text name
        text phone
        citext email
        numeric credit_limit
```

```
        timestamptz deleted_at
    }
    SUPPLIERS {
        uuid id PK
        uuid company_id FK
        text code UK
        text name
        text phone
        smallint payment_terms_days
        timestamptz deleted_at
    }
    CATEGORIES {
        uuid id PK
        uuid company_id FK
        uuid parent_id FK
        text name UK
        timestamptz deleted_at
    }
    PRODUCTS {
        uuid id PK
        uuid company_id FK
        uuid category_id FK
        text sku UK
        text barcode
        text name
        text unit
        boolean is_stock_tracked
        numeric default_tax_rate "V1.1"
        numeric reorder_point
        boolean is_active
        timestamptz deleted_at
    }
    PRODUCT_PRICES {
        uuid id PK
        uuid company_id FK
        uuid product_id FK
        numeric unit_price
        numeric cost_price
        timestamptz valid_from
        timestamptz valid_to
        uuid created_by FK
    }
    PRODUCT_STOCK {
        uuid product_id PK
        uuid company_id FK
        numeric qty_on_hand
        numeric avg_cost "V1.1 - CMP"
        numeric stock_value "V1.1 - genere"
        timestamptz last_movement_at
        timestamptz recomputed_at
    }
    SALES {
        uuid id PK
        uuid company_id FK
        uuid customer_id FK
        text sale_no UK
        sale_status status
        timestamptz sold_at
        numeric subtotal
        numeric discount_total
        numeric tax_total "V1.1"
```

```
        numeric total
        numeric paid_total
        text cancel_reason
        timestamptz confirmed_at
        timestamptz cancelled_at
        uuid created_by FK
    }
    SALE_ITEMS {
        uuid id PK
        uuid company_id FK
        uuid sale_id FK
        uuid product_id FK
        uuid price_id FK
        numeric qty
        numeric unit_price
        numeric unit_cost
        numeric discount
        numeric tax_rate "V1.1"
        numeric tax_amount "V1.1 - genere"
        numeric net_amount "genere"
        numeric line_total "genere"
    }
    SALE_RETURNS {
        uuid id PK
        uuid company_id FK
        uuid sale_id FK
        text return_no UK
        text reason
        timestamptz returned_at
        numeric total_refund
        numeric refunded_total "V1.1"
        return_refund_status refund_status "V1.1"
        uuid created_by FK
    }
    SALE_RETURN_ITEMS {
        uuid id PK
        uuid company_id FK
        uuid return_id FK
        uuid sale_item_id FK
        numeric qty
        numeric refund_amount
        numeric tax_refund "V1.1"
        numeric unit_cost "V1.1"
    }
    SALE_RETURN_REFUNDS {
        uuid id PK
        uuid company_id FK
        uuid sale_return_id FK
        uuid payment_id FK
        numeric amount
        timestamptz created_at
    }
    INVENTORY_MOVEMENTS {
        uuid id PK
        uuid company_id FK
        uuid product_id FK
        movement_type type
        numeric qty_delta
        numeric unit_cost
        text ref_type
        uuid ref_id
```

```
        uuid reversal_of_movement_id FK "V1.1"
        uuid supplier_id FK
        text note
        timestamptz occurred_at
        uuid created_by FK
    }
    EXPENSE_CATEGORIES {
        uuid id PK
        uuid company_id FK
        text name UK
        expense_kind kind
    }
    EXPENSES {
        uuid id PK
        uuid company_id FK
        uuid category_id FK
        uuid supplier_id FK
        text label
        numeric amount
        date spent_at
        timestamptz deleted_at
        uuid created_by FK
    }
    DEBTS {
        uuid id PK
        uuid company_id FK
        party_type party_type
        uuid customer_id FK
        uuid supplier_id FK
        uuid sale_id FK
        uuid expense_id FK
        numeric principal
        numeric amount_paid
        numeric balance "genere"
        date due_date
        debt_status status
        timestamptz closed_at
    }
    PAYMENTS {
        uuid id PK
        uuid company_id FK
        cash_direction direction
        pay_method method
        numeric amount
        timestamptz paid_at
        uuid sale_id FK
        uuid expense_id FK
        uuid sale_return_id FK "V1.1"
        text reference
        uuid created_by FK
    }
    PAYMENT_ALLOCATIONS {
        uuid id PK
        uuid company_id FK
        uuid payment_id FK
        uuid debt_id FK
        numeric amount
    }
    NOTIFICATIONS {
        uuid id PK
        uuid company_id FK
```

```
        uuid user_id FK
        text type
        text title
        text body
        jsonb data
        timestamptz read_at
    }
    AUDIT_LOGS {
        bigint id PK
        uuid company_id FK
        uuid actor_user_id FK
        text action
        text entity_table
        uuid entity_id
        jsonb diff
        inet ip_address
        timestamptz occurred_at
    }
    BUSINESS_EVENTS {
        bigint id PK
        uuid company_id FK
        text event_type
        text aggregate_type
        uuid aggregate_id
        jsonb payload
        timestamptz occurred_at
        timestamptz published_at
        smallint attempts
    }
    DAILY_SALES_FACTS {
        uuid company_id PK
        date day PK
        uuid product_id PK
        uuid category_id
        numeric qty_sold
        numeric returned_qty
        numeric net_qty_sold
        numeric gross_revenue
        numeric discount_total
        numeric tax_total
        numeric refund_net_amount
        numeric net_revenue
        numeric cogs_gross
        numeric cogs_returned
        numeric cogs_net
        integer sale_count
        timestamptz built_at
    }
    ML_FEATURE_SNAPSHOTS {
        bigint id PK
        uuid company_id FK
        uuid product_id FK
        date as_of
        jsonb features
        jsonb labels
        timestamptz created_at
    }
    FORECASTS {
        uuid id PK
        uuid company_id FK
        uuid product_id FK
```

```
        forecast_type forecast_type
        date as_of
        date horizon_date
        numeric yhat
        numeric yhat_lower
        numeric yhat_upper
        text model_name
        text model_version
        jsonb metadata
        timestamptz generated_at
    }
    INSIGHTS {
        uuid id PK
        uuid company_id FK
        insight_type insight_type
        insight_severity severity
        text title
        text body
        text entity_table
        uuid entity_id
        jsonb evidence
        text dedupe_key UK
        text model_name
        text model_version
        insight_status status
        timestamptz generated_at
        timestamptz valid_until
    }
```

4. Dictionnaire de données V1.1

Conventions inchangées : PK uuid via gen_random_uuid() ; created_at timestamptz NOT NULL DEFAULT now() et updated_at par trigger ; company_id uuid NOT NULL sur toute table métier avec UNIQUE (company_id, id) comme cible des FK composites ; montants numeric(14,2), quantités numeric(14,3), coûts unitaires numeric(14,4) (précision nécessaire au CMP), taux numeric(6,4) ; jamais de flottant. La section 4.1 recense les 30 tables ; 4.2 à 4.12 détaillent au niveau colonne les tables nouvelles ou amendées. Pour les tables inchangées, le détail colonne par colonne de l'annexe V1 reste la référence et n'est pas dupliqué ici.

4.1 Recensement des 30 tables

Table	Statut	Rôle	Vérité	RLS	Colonnes
companies	V1	Racine du tenant	Source	non*	id, name, slug, currency, timezone, settings, created_at, updated_at, deleted_at
users	V1	Comptes d'accès, 1 par entreprise	Source	oui	id, company_id, email, password_hash, full_name, is_active, last_login_at, deleted_at
roles	V1	Rôles RBAC par tenant	Source	oui	id, company_id, name, description, is_system
permissions	V1	Catalogue global des droits	Source	non*	id, code, resource, action
user_roles	V1	Liaison utilisateur ↔ rôle	Source	oui	user_id, role_id, company_id, granted_at
role_permissions	V1	Liaison rôle ↔ permission	Source	oui	role_id, permission_id, company_id
customers	V1	Clients	Source	oui	id, company_id, code, name, phone, email, credit_limit, deleted_at
suppliers	V1	Fournisseurs	Source	oui	id, company_id, code, name, phone, payment_terms_days, deleted_at
categories	V1	Arbre de familles produit	Source	oui	id, company_id, parent_id, name, deleted_at
products	AMENDÉE	Référentiel article	Source	oui	+ default_tax_rate
product_prices	V1	Historique tarifaire par intervalles	Source	oui	id, company_id, product_id, unit_price, cost_price, valid_from, valid_to, created_by
product_stock	AMENDÉE	État courant + CMP	Projection	oui	+ avg_cost, stock_value, recomputed_at
sales	AMENDÉE	Racine d'agrégat de la vente	Source	oui	+ tax_total, total redéfini

Table	Statut	Rôle	Vérité	RLS	Colonnes
sale_items	AMENDÉE	Lignes de vente, snapshots	Source	oui	+ tax_rate, tax_amount, net_amount
sale_returns	AMENDÉE	En-tête de retour client	Source	oui	+ refunded_total, refund_status
sale_return_items	AMENDÉE	Lignes de retour	Source	oui	+ tax_refund, unit_cost
sale_return_refunds	NOUVELLE	Lettrage retour ↔ paiement	Source	oui	id, company_id, sale_return_id, payment_id, amount, created_at
inventory_movements	AMENDÉE	Journal de stock append-only	Source	oui	+ reversal_of_movement_id
expense_categories	V1	Axes de charges	Source	oui	id, company_id, name, kind
expenses	V1	Charges	Source	oui	id, company_id, category_id, supplier_id, label, amount, spent_at, deleted_at, created_by
debts	V1	Créances et dettes	Source	oui	id, company_id, party_type, customer_id, supplier_id, sale_id, expense_id, principal, amount_paid, balance, due_date, status, closed_at
payments	AMENDÉE	Flux de trésorerie	Source	oui	+ sale_return_id
payment_allocations	V1	Lettrage paiement ↔ dette	Source	oui	id, company_id, payment_id, debt_id, amount, created_at
notifications	V1	Alertes utilisateur	Source	oui	id, company_id, user_id, type, title, body, data, read_at
audit_logs	RLS AJOUTÉE	Journal de conformité	Source	**oui**	id, company_id, actor_user_id, action, entity_table, entity_id, diff, ip_address, occurred_at
business_events	V1	Outbox transactionnelle	Source	oui	id, company_id, event_type, aggregate_type, aggregate_id, payload, occurred_at, published_at, attempts
daily_sales_facts	MV → TABLE	Agrégat jour × produit	Projection	**oui**	voir 4.10
ml_feature_snapshots	CRÉÉE	Features figées par as_of	Projection	oui	id, company_id, product_id, as_of, features, labels, created_at
forecasts	NOUVELLE	Prévisions persistées	Projection	oui	voir 4.11

Table	Statut	Rôle	Vérité	RLS	Colonnes
insights	NOUVELLE	Anomalies et recommandations	Projection	oui	voir 4.12

* companies et permissions ne sont pas tenant-scoped : la première *est* le tenant, la seconde est un catalogue global en lecture seule pour l'application. Une politique RLS USING (id = current_setting('app.company_id', true)::uuid) est néanmoins posée sur companies pour éviter l'énumération des tenants.

4.2 products AMENDÉE

Une seule colonne ajoutée. Aucune notion de variante. default_tax_rate est un **paramètre de saisie**, jamais une source de recalcul : sa valeur est copiée dans sale_items.tax_rate à la confirmation et sa modification ultérieure n'affecte aucune vente passée.

Colonne	Type	Null	Contraintes / notes
default_tax_rate	numeric(6,4)	NON	V1.1 défaut 0 ; CHECK (default_tax_rate >= 0 AND default_tax_rate < 1) ; taux décimal (0.18 = 18 %)
(autres colonnes)	—	—	inchangées : id, company_id, category_id, sku, barcode, name, unit, is_stock_tracked, reorder_point, is_active, deleted_at

4.3 product_stock AMENDÉE — PROJECTION CMP

Reste une **projection** sans vérité propre, désormais porteuse du CMP courant. Elle n'existe que pour les articles is_stock_tracked = true : la cardinalité products → product_stock passe donc de 1:1 à 1:0..1. Le trigger de mouvement la met à jour sous verrou de ligne ; un job nocturne la reconstruit depuis inventory_movements et journalise tout écart dans audit_logs.

Colonne	Type	Null	Contraintes / notes
product_id	uuid	NON	PK ; FK composite (company_id, product_id) → products
company_id	uuid	NON	FK → companies ; UNIQUE (company_id, product_id)
qty_on_hand	numeric(14,3)	NON	défaut 0 ; **CHECK (qty_on_hand >= 0)** — stock négatif interdit sans dérogation
avg_cost	numeric(14,4)	NON	V1.1 défaut 0 ; CHECK ≥ 0 ; **CMP courant**, recalculé à chaque entrée, inchangé à chaque sortie
stock_value	numeric(14,2)	NON	V1.1 **GENERATED ALWAYS AS** round(qty_on_hand * avg_cost, 2) STORED — ne peut pas diverger
last_movement_at	timestamptz	OUI	horodatage métier du dernier mouvement appliqué
recomputed_at	timestamptz	NON	RENOMMÉE (ex-refreshed_at) défaut now() ; date de la dernière reconstruction complète

Index : PK(product_id) ; UNIQUE(company_id, product_id) ; (company_id, qty_on_hand) pour les alertes de seuil ; (company_id) WHERE qty_on_hand = 0 pour les ruptures.

4.4 sales AMENDÉE — TAXES FIGÉES

Colonne	Type	Null	Contraintes / notes
subtotal	numeric(14,2)	NON	Σ qty × unit_price des lignes, **hors taxe et avant remise** ; maintenu par trigger ; CHECK ≥ 0
discount_total	numeric(14,2)	NON	Σ des remises de ligne ; CHECK (0 <= discount_total AND discount_total <= subtotal)
tax_total	numeric(14,2)	NON	V1.1 défaut 0 ; CHECK ≥ 0 ; Σ sale_items.tax_amount, maintenu par trigger
total	numeric(14,2)	NON	REDÉFINI **GENERATED ALWAYS AS** (subtotal − discount_total + tax_total) STORED — l'égalité comptable est portée par la base
paid_total	numeric(14,2)	NON	défaut 0 ; maintenu par trigger depuis payments (inflow rattachés à la vente)
(autres colonnes)	—	—	inchangées : id, company_id, customer_id, sale_no, status, sold_at, cancel_reason, cancelled_at, confirmed_at, created_by

Index : inchangés, dont le partiel (company_id, sold_at) WHERE status = 'confirmed' qui sert tous les rapports et la reconstruction analytique.

4.5 sale_items AMENDÉE — SNAPSHOT FISCAL

Les taxes **font partie du périmètre V1.1**, sous une forme volontairement minimale : un taux unique par ligne, figé à la confirmation. Il n'existe pas de moteur fiscal, pas de table de règles, pas de territorialité, pas d'exonération conditionnelle. La base garantit seulement qu'un montant émis ne peut plus changer.

Colonne	Type	Null	Contraintes / notes
unit_price	numeric(14,2)	NON	snapshot HT ; immuable après confirmation (trigger)
unit_cost	numeric(14,4)	NON	NON NULLABLE snapshot du **CMP applicable à l'instant de la sortie**, recopié depuis le mouvement ; base du COGS historique
discount	numeric(14,2)	NON	défaut 0 ; CHECK (discount >= 0 AND discount <= qty * unit_price)
tax_rate	numeric(6,4)	NON	V1.1 défaut 0 ; CHECK >= 0 AND < 1 ; copié de products.default_tax_rate à la confirmation, jamais relu ensuite
net_amount	numeric(14,2)	NON	V1.1 **GENERATED** (qty * unit_price − discount) STORED — base taxable de la ligne
tax_amount	numeric(14,2)	NON	V1.1 **GENERATED** round((qty * unit_price − discount) * tax_rate, 2) STORED
line_total	numeric(14,2)	NON	REDÉFINI **GENERATED** (qty * unit_price − discount + round((qty * unit_price − discount) * tax_rate, 2)) STORED — montant TTC de la ligne
(autres colonnes)	—	—	inchangées : id, company_id, sale_id, product_id, price_id, qty

4.6 sale_returns AMENDÉE

Colonne	Type	Null	Contraintes / notes
total_refund	numeric(14,2)	NON	montant **remboursable** TTC ; Σ des refund_amount de lignes ; maintenu par trigger ; CHECK ≥ 0
refunded_total	numeric(14,2)	NON	V1.1 défaut 0 ; Σ des sale_return_refunds.amount, maintenu par trigger ; CHECK (refunded_total <= total_refund) — **le double remboursement est impossible**
refund_status	return_refund_status	NON	V1.1 ENUM (pending, partial, settled, none_due) ; dérivé par trigger du couple (total_refund, refunded_total)
(autres colonnes)	—	—	inchangées : id, company_id, sale_id, return_no, reason, returned_at, created_by

Statut de la notion d'avoir client : explicitement hors V1. Il n'existe pas de table `credit_notes` et aucun solde créditeur client n'est représentable. Un retour se règle soit par un remboursement en trésorerie (`payments.direction = 'outflow'`), soit par une extinction de dette existante via `payment_allocations`, soit par rien du tout (`refund_status = 'none_due'`). L'ancien libellé « remboursement ou avoir » de la V1 est retiré de la conception.

4.7 sale_return_items AMENDÉE · sale_return_refunds NOUVELLE

Pourquoi une table de lettrage plutôt qu'une simple FK : `payments.sale_return_id` seul ne permettrait pas de rembourser deux retours par un unique versement, ni d'échelonner un remboursement en deux versements. La structure retenue est symétrique de `payment_allocations`, déjà éprouvée dans le modèle. `payments.sale_return_id` est conservé comme **contexte direct** (lecture rapide, cohérent avec `sale_id` et `expense_id`), le lettrage porte la relation N:N.

Table.colonne	Type	Null	Contraintes / notes
sale_return_items.qty	numeric(14,3)	NON	CHECK > 0 ; trigger : Σ des retours d'une ligne ≤ sale_items.qty
sale_return_items.refund_amount	numeric(14,2)	NON	CHECK ≥ 0 ; TTC, calculé au prix et au taux historiques de la ligne : qty × (unit_price − discount/qty) × (1 + tax_rate), arrondi à 2 décimales
sale_return_items.tax_refund	numeric(14,2)	NON	V1.1 défaut 0 ; part de taxe incluse dans refund_amount ; CHECK 0 <= tax_refund <= refund_amount
sale_return_items.unit_cost	numeric(14,4)	NON	V1.1 recopié de sale_items.unit_cost : le retour réintègre le stock **au coût auquel il en était sorti**, pas au CMP du jour
sale_return_refunds.sale_return_id	uuid	NON	FK composite → sale_returns ON DELETE RESTRICT
sale_return_refunds.payment_id	uuid	NON	FK composite → payments ON DELETE CASCADE ; UNIQUE (company_id, payment_id, sale_return_id)
sale_return_refunds.amount	numeric(14,2)	NON	CHECK > 0 ; triggers : Σ par retour ≤ total_refund, Σ par paiement ≤ payments.amount, et payments.direction doit valoir outflow

Index : (company_id, sale_return_id) — répond à « quel remboursement pour ce retour » ; (company_id, payment_id) — répond à « quel retour a provoqué cette sortie de trésorerie ».

4.8 inventory_movements AMENDÉE — MOUVEMENTS INVERSES

Toujours append-only : ni UPDATE ni DELETE, garantis par des règles DO INSTEAD NOTHING. Le mouvement d'origine reste strictement immuable ; son annulation crée une **nouvelle** ligne qui le désigne.

Colonne	Type	Null	Contraintes / notes
type	movement_type	NON	ENUM inchangé (initial, purchase_in, sale_out, customer_return_in, supplier_return_out, adjustment, loss)
qty_delta	numeric(14,3)	NON	CHECK <> 0 ; signe contraint par type (entrées > 0, sorties < 0, adjustment libre)
unit_cost	numeric(14,4)	NON	NON NULLABLE CHECK ≥ 0 ; **entrées :** coût d'acquisition fourni ; **sorties :** estampillé au CMP par le trigger ; **retours et inverses :** coût du mouvement d'origine
ref_type	text	OUI	ÉTENDU CHECK IN (sale_item, sale_return_item, manual, import, initial)
ref_id	uuid	OUI	idempotence par index unique partiel (company_id, ref_type, ref_id) WHERE reversal_of_movement_id IS NULL — voir note ci-dessous
reversal_of_movement_id	uuid	OUI	V1.1 FK composite auto-référente (company_id, reversal_of_movement_id) → inventory_movements (company_id, id) ON DELETE RESTRICT ; CHECK (reversal_of_movement_id <> id) ; **UNIQUE (company_id, reversal_of_movement_id)**
(autres colonnes)	—	—	inchangées : id, company_id, product_id, supplier_id, note, occurred_at, created_by

Articulation avec ref_type / ref_id, et pourquoi elle est nécessaire. Un mouvement inverse conserve les ref_type et ref_id de l'objet métier d'origine — c'est ce qui permet de retrouver, à partir d'une ligne de vente, à la fois sa sortie et son annulation. Mais l'index d'idempotence V1 portait sur (company_id, ref_type, ref_id) sans condition : il aurait rejeté l'inverse comme un doublon. Il devient donc **partiel** sur reversal_of_movement_id IS NULL, ce qui préserve exactement la propriété recherchée : au plus un mouvement *direct* par objet métier. L'unicité de l'annulation est portée séparément, et plus fortement, par UNIQUE (company_id, reversal_of_movement_id) : **un mouvement ne peut être annulé qu'une seule fois**, quel que soit le nombre de fois où l'API est appelée. Une double annulation de vente est rejetée par la base, pas par une vérification applicative.

Trois règles complémentaires sont portées par un trigger de validation : (a) le mouvement inverse doit viser le même product_id que sa cible ; (b) son qty_delta doit être exactement l'opposé de celui de la cible ; (c) on ne peut pas annuler un mouvement qui est lui-même une annulation. Le type de l'inverse reste sémantique (customer_return_in pour une sortie de vente annulée, adjustment pour une correction d'inventaire), ce qui conserve la lisibilité des rapports de mouvements.

Index : UNIQUE(company_id, id) ; (company_id, product_id, occurred_at DESC) ; BRIN(occurred_at) ; UNIQUE partiel d'idempotence ; UNIQUE(company_id, reversal_of_movement_id) ; (company_id, ref_type, ref_id) non unique pour la lecture.

4.9 payments AMENDÉE

Colonne	Type	Null	Contraintes / notes
direction	cash_direction	NON	ENUM (inflow, outflow) ; le sens n'est jamais porté par un montant négatif
amount	numeric(14,2)	NON	CHECK > 0
sale_return_id	uuid	OUI	V1.1 FK composite → sale_returns ON DELETE RESTRICT
contexte direct	—	—	CHECK ÉTENDU au plus un de (sale_id, expense_id, sale_return_id) renseigné : CHECK (num_nonnulls(sale_id, expense_id, sale_return_id) <= 1)
cohérence de sens	—	—	V1.1 CHECK (sale_return_id IS NULL OR direction = 'outflow') — un remboursement de retour ne peut pas être un encaissement
(autres colonnes)	—	—	inchangées : id, company_id, method, paid_at, sale_id, expense_id, reference, note, created_by

4.10 daily_sales_facts VUE MATÉRIALISÉE → TABLE RLS

Grain : entreprise × jour de vente × produit. **Table réelle**, jamais une vue matérialisée : une MV n'est pas soumise aux politiques RLS de ses tables sources, l'exposer directement à l'API laissait un chemin de lecture sans barrière tenant. Reste une projection intégralement reconstruisible ; aucune décision métier ne s'appuie sur elle.

Colonne	Type	Définition exacte
company_id, day, product_id	uuid, date, uuid	**PK composite.** day = jour de sales.sold_at converti dans le fuseau de l'entreprise ; les retours sont rattachés au jour de la **vente d'origine**, pas au jour du retour
category_id	uuid	dénormalisé depuis products à la construction ; évite la jointure sur les agrégats par famille
qty_sold	numeric(14,3)	quantité vendue **brute** : Σ sale_items.qty des ventes confirmed
returned_qty	numeric(14,3)	quantité retournée, **pré-agrégée par sale_item_id** avant jointure
net_qty_sold	numeric(14,3)	qty_sold − returned_qty ; quantité réellement écoulée
gross_revenue	numeric(14,2)	Σ qty × unit_price : chiffre d'affaires HT **avant** remise et **hors** retour
discount_total	numeric(14,2)	Σ sale_items.discount : remises réellement appliquées
tax_total	numeric(14,2)	Σ sale_items.tax_amount ; isolé pour ne jamais polluer une mesure de revenu
refund_net_amount	numeric(14,2)	Σ (refund_amount − tax_refund) des retours : montant HT rendu au client
net_revenue	numeric(14,2)	gross_revenue − discount_total − refund_net_amount ; **HT, net de remise et de retour**. La formule correspond désormais au nom : la V1 appelait ainsi une mesure qui ignorait les retours et incluait des lignes dupliquées
cogs_gross	numeric(14,2)	Σ qty × sale_items.unit_cost : coût des marchandises sorties, au CMP figé à la vente
cogs_returned	numeric(14,2)	Σ returned_qty × sale_return_items.unit_cost : coût réintégré
cogs_net	numeric(14,2)	cogs_gross − cogs_returned ; c'est cette mesure qui se retranche de net_revenue pour obtenir la marge
sale_count	integer	COUNT(DISTINCT sale_id) — nombre de tickets contenant l'article ; distingue « beaucoup vendu » de « vendu souvent ». La duplication V1 le gonflait mécaniquement
built_at	timestamptz	date de construction de la ligne ; sert au contrôle de fraîcheur et au diagnostic

Contraintes : FK composites vers companies et products ; CHECK de non-négativité sur les quantités ; CHECK (net_qty_sold = qty_sold − returned_qty). **Index :** PK(company_id, day, product_id) ; (company_id, day) ; (company_id, product_id, day) ; (company_id, category_id, day) ; BRIN(day). **RLS :** activée et forcée.

Conséquence du rattachement des retours au jour de la vente. Un retour enregistré aujourd'hui modifie une ligne de fait datée d'il y a trois semaines. La reconstruction doit donc être **par plage réaffectée**, pas seulement incrémentale sur la veille : le job lit `business_events` depuis le dernier passage, en déduit l'ensemble des jours de vente touchés (ventes créées, confirmées, annulées, retournées) et réécrit ces jours-là en DELETE … INSERT sous une seule transaction par entreprise. C'est un choix assumé : il garantit qu'une ligne de fait décrit la *réalité finale* d'une journée de vente, au prix d'un retraitement du passé. L'alternative — rattacher le retour à son propre jour — donnerait des séries plus stables mais un `net_revenue` journalier faux pour la prévision de demande.

4.11 forecasts NOUVELLE

Pourquoi cette table : sans elle, afficher « rupture probable dans 5 jours » exigerait d'exécuter un modèle dans le cycle de requête HTTP. La table découple totalement inférence et lecture : le pipeline écrit, l'API lit. Écrite exclusivement par le processus analytique, jamais par un endpoint applicatif.

Colonne	Type	Null	Contraintes / notes
id	uuid	NON	PK
company_id	uuid	NON	FK → companies ; UNIQUE (company_id, id)
product_id	uuid	OUI	FK composite → products ; NULL = prévision agrégée à l'entreprise (CA global, trésorerie)
forecast_type	forecast_type	NON	ENUM (demand_qty, revenue, stockout_date, cash_balance)
as_of	date	NON	date de référence : état d'information utilisé. CHECK (horizon_date >= as_of)
horizon_date	date	NON	date prévue
yhat	numeric(14,4)	NON	valeur prédite
yhat_lower / yhat_upper	numeric(14,4)	OUI	intervalle si le modèle en produit ; CHECK (yhat_lower IS NULL OR yhat_upper IS NULL OR yhat_lower <= yhat_upper)
model_name / model_version	text	NON	traçabilité : quelle version a produit ce chiffre ; indispensable au backtest
metadata	jsonb	NON	défaut '{}' : métriques d'entraînement, features clés, fenêtre utilisée
generated_at	timestamptz	NON	défaut now()

Idempotence : UNIQUE NULLS NOT DISTINCT (company_id, forecast_type, product_id, as_of, horizon_date, model_version) — syntaxe PostgreSQL 15+, indispensable ici puisque product_id est nullable ; un rejeu du pipeline fait un ON CONFLICT DO UPDATE au lieu de dupliquer. **Index :** (company_id, product_id, forecast_type, horizon_date) — lecture UI ; partiel (company_id, as_of DESC). **RLS :** activée et forcée.

4.12 insights NOUVELLE

Anomalies, observations et recommandations persistées, qu'elles viennent d'un modèle ou d'une règle déterministe. evidence porte les chiffres qui expliquent l'insight : l'UI doit pouvoir justifier une alerte sans relancer un calcul.

Colonne	Type	Null	Contraintes / notes
id	uuid	NON	PK ; UNIQUE (company_id, id)
company_id	uuid	NON	FK → companies
insight_type	insight_type	NON	ENUM (stockout_risk, slow_mover, demand_spike, margin_drop, expense_anomaly, cash_shortfall, debt_overdue, cashier_anomaly)
severity	insight_severity	NON	ENUM (info, warning, critical) ; pilote le tri et la notification
title	text	NON	CHECK longueur 1–160 ; phrase courte prête à l'affichage
body	text	OUI	explication et recommandation
entity_table / entity_id	text / uuid	OUI	entité concernée. CHECK IN (products, customers, suppliers, expense_categories, sales, users) ; **référence souple volontairement non FK** — une FK polymorphe n'est pas exprimable, et l'insight doit survivre à l'archivage de sa cible. CHECK (num_nonnulls(entity_table, entity_id) <> 1) : les deux ou aucun
evidence	jsonb	NON	défaut '{}' : séries, seuils, écarts, période observée
dedupe_key	text	NON	clé de déduplication stable, ex. stockout_risk:<product_id>:2026-W12 ; **évite de republier chaque nuit la même alerte**
model_name / model_version	text	OUI	NULL si l'insight vient d'une règle métier et non d'un modèle — distinction utile au diagnostic
status	insight_status	NON	ENUM (new, read, acknowledged, dismissed, resolved) ; défaut 'new' ; **seule colonne modifiable par l'application**
read_at / resolved_at	timestamptz	OUI	CHECK croisés avec status
generated_at	timestamptz	NON	défaut now()
valid_until	timestamptz	OUI	péremption : au-delà l'insight n'est plus affiché ; purge par job

Idempotence : index unique partiel (company_id, dedupe_key) WHERE status <> 'dismissed' — un rejeu met à jour l'insight vivant ; un insight écarté par l'utilisateur ne bloque pas une réémission future si le problème réapparaît. **Index :** (company_id, status, severity, generated_at DESC) — liste UI ; (company_id, entity_table, entity_id) ; GIN sur evidence si recherche par champ. **RLS :** activée et forcée.

Frontière avec `notifications` : un insight est un *fait analytique* daté et persistant, qui garde son évidence même lu. Une notification est un *message à un destinataire*, jetable après lecture. Un insight `critical` déclenche l'écriture d'une notification ; l'inverse n'existe pas. Les deux tables ne sont pas fusionnées parce que leurs cycles de vie et leurs volumes diffèrent d'un ordre de grandeur.

5. Règles métier critiques

Chaque opération est une transaction unique, ouverte par `SET LOCAL app.company_id`. Niveau `READ COMMITTED`, suffisant grâce aux verrous de ligne explicites pris dans un ordre déterministe (`product_id` croissant) — ce qui élimine les interblocages entre deux caisses simultanées.

5.1 Confirmation d'une vente

1. Vérifier `status = 'draft'` et au moins une ligne ; verrouiller la vente `FOR UPDATE`.
2. Verrouiller les lignes `product_stock` des articles suivis, ordonnées par `product_id`.
3. **Figer les snapshots :** `unit_price` et `price_id` depuis `product_prices` où `valid_to IS NULL` ; `tax_rate` depuis `products.default_tax_rate`. `net_amount`, `tax_amount` et `line_total` sont générés par la base.
4. Insérer un mouvement `sale_out` par ligne d'article suivi, `qty_delta = −qty`, `ref_type = 'sale_item'`, `ref_id = sale_item.id`. Le trigger estampille `unit_cost` au CMP courant, refuse le stock négatif et met à jour la projection.
5. Recopier `inventory_movements.unit_cost` dans `sale_items.unit_cost` : c'est le COGS historique de la ligne, définitivement figé. Les articles non suivis prennent `unit_cost = 0`.
6. Recalculer `sales.subtotal`, `discount_total`, `tax_total` ; `total` est généré. Passer `status = 'confirmed'`, `confirmed_at = now()`.
7. Volet financier : `payments` (inflow) pour la part encaissée, `debts` pour la part à crédit, avec contrôle du plafond client.
8. `business_events` : `sale.confirmed`, dans la même transaction (outbox). `COMMIT`.

Cohérence arithmétique garantie : `sous-total − remises + taxes = total` n'est pas vérifié par le code mais *produit* par une colonne générée. Aucune écriture ne peut créer une vente déséquilibrée.

5.2 Annulation d'une vente

1. Verrouiller la vente. Refuser si `status <> 'confirmed'`, ou s'il existe un retour, ou un paiement encaissé non remboursé — l'annulation n'est pas un substitut au retour.
2. Pour chaque mouvement `sale_out` de la vente : insérer un mouvement inverse `customer_return_in`, `qty_delta = −qty_delta_origine`, `unit_cost = unit_cost_origine`, `reversal_of_movement_id = id_origine`, `ref_type`/`ref_id` repris de l'origine.
3. **Idempotence :** un second appel viole `UNIQUE (company_id, reversal_of_movement_id)` et échoue au niveau SQL. L'API traduit ce conflit en réponse « déjà annulée », sans effet de bord.
4. Dette liée : si `amount_paid = 0`, `status = 'written_off'` et `closed_at = now()` ; sinon refus, la dette doit d'abord être traitée financièrement.
5. `status = 'cancelled'`, `cancelled_at`, `cancel_reason` obligatoire. Événement `sale.cancelled`.

Le mouvement d'origine n'est ni modifié ni supprimé — les règles `DO INSTEAD NOTHING` le rendent impossible même pour un UPDATE direct en base.

5.3 Retour client

1. Vente `confirmed` exigée ; `returned_at >= sold_at`.
2. Par ligne : `qty` retournée ≤ `sale_items.qty` moins les retours déjà enregistrés (trigger sous verrou). `refund_amount` et `tax_refund` calculés au prix et au taux **historiques**. `unit_cost` recopié de `sale_items.unit_cost`.

3. Mouvement `customer_return_in`, `qty_delta = +qty`, `unit_cost = sale_return_items.unit_cost`, `ref_type = 'sale_return_item'`. **Sans** `reversal_of_movement_id` : un retour est un fait métier autonome, pas l'annulation d'un mouvement.
4. `total_refund` et `refund_status` recalculés par trigger.
5. Règlement, au choix du gérant : remboursement en trésorerie (5.4), ou extinction d'une dette client existante par `payment_allocations`, ou rien (`none_due`).
6. Événement `sale.returned`, puis retraitement analytique du jour de vente concerné.

Traitement du coût au retour — règle explicite. La marchandise revient en stock **au coût auquel elle en est sortie** (`sale_items.unit_cost`), et non au CMP du jour. Deux raisons : le COGS historique de la vente reste exactement compensé, donc la marge nette de la période est juste ; et un retour ne doit pas créer de profit ou de perte fictif par simple variation du CMP entre la vente et le retour. Conséquence assumée : cette entrée entre dans la formule du CMP comme toute autre entrée, et peut donc faire légèrement varier le CMP courant si celui-ci a changé entre-temps. C'est le comportement normal d'une valorisation en coût moyen ; il n'existe pas de solution exacte sans passer à un suivi par couches (FIFO), explicitement hors V1.

5.4 Remboursement d'un retour

1. Verrouiller `sale_returns FOR UPDATE` ; calculer le reste dû `total_refund − refunded_total`.
2. Insérer `payments` avec `direction = 'outflow'` et `sale_return_id` renseigné — le CHECK rejette tout `inflow`.
3. Insérer `sale_return_refunds` ; les triggers refusent un montant qui dépasserait soit le reste dû du retour, soit le montant du paiement.
4. `refunded_total` et `refund_status` recalculés par trigger.
5. Événement `sale_return.refunded`.

Les deux questions du cahier des charges sont désormais répondues par une jointure : « quel remboursement correspond à ce retour » = `sale_return_refunds` filtré sur `sale_return_id` ; « quel retour a provoqué cette sortie » = le même index lu par `payment_id`. Un remboursement partiel puis complémentaire est représentable ; un remboursement supérieur au remboursable est rejeté ; un remboursement dans le mauvais sens financier est rejeté.

5.5 Dettes et paiements

- **Paiement direct :** `payments` avec `sale_id` ou `expense_id`, sans dette ni allocation.
- **Paiement partiel :** une allocation d'un montant inférieur au solde ; `debts.status` devient `partially_paid` par trigger.
- **Paiement réparti sur plusieurs dettes :** N allocations pour un même `payment_id` ; le trigger contrôle que Σ allocations ≤ `payments.amount` et, par dette, que Σ ≤ `principal`, chaque dette étant verrouillée `FOR UPDATE`.
- **Dette soldée :** `balance` (colonne générée) atteint 0 → `status = 'paid'`, `closed_at` renseigné.
- **Plafond de crédit :** à la création d'une dette client, Σ des soldes ouverts du client doit rester sous `customers.credit_limit` quand il est renseigné.
- **Abandon :** `written_off` avec motif en `audit_logs` ; `principal` n'est jamais réécrit.

5.6 Mouvement de stock et CMP

Un seul point d'entrée : l'insertion dans `inventory_movements`. Aucun code applicatif n'écrit `product_stock`.

```
ENTREE (qty_delta > 0)                    SORTIE (qty_delta < 0)
-----------------------------------       -----------------------------------
unit_cost fourni, obligatoire             unit_cost := avg_cost courant (estampille)
new_qty := qty_on_hand + qty_delta        new_qty := qty_on_hand + qty_delta
                                          SI new_qty < 0 -> EXCEPTION (rejet atomique)
new_avg :=
  (qty_on_hand * avg_cost
   + qty_delta * unit_cost)               new_avg := avg_cost           (inchange)
  / new_qty
SI new_qty = 0 -> new_avg := avg_cost     stock_value recalcule (colonne generee)
  (dernier CMP conserve, pas de 0/0)
```

- **Verrou :** la ligne `product_stock` est verrouillée avant lecture du CMP, ce qui sérialise deux ventes simultanées du même article et rend le calcul exact sous concurrence.
- **Articles non suivis :** `is_stock_tracked = false` → toute insertion de mouvement est refusée, aucune ligne de projection n'existe, le CMP n'a pas de sens pour eux.
- **Idempotence :** un même `(ref_type, ref_id)` ne peut produire qu'un mouvement direct ; un même mouvement ne peut être annulé qu'une fois.
- **COGS :** obtenu en lisant `unit_cost` figé sur les mouvements ou les lignes de vente — jamais en relisant un coût d'aujourd'hui.

5.7 Reconstruction du stock et du CMP

`product_stock` étant une projection, elle doit être entièrement recalculable. La quantité est une simple somme, mais **le CMP ne l'est pas** : il dépend de l'ordre des mouvements. La reconstruction rejoue donc les mouvements par `(occurred_at, id)` croissant et applique la même récurrence que le trigger. La fonction de reconstruction et le trigger partagent volontairement la même formule, appelée depuis une fonction unique, pour qu'aucune divergence ne soit possible.

- **Ordre canonique :** `ORDER BY occurred_at, id`. Un mouvement antidaté inséré après coup change donc le CMP recalculé — le job nocturne journalise l'écart dans `audit_logs` et adopte la valeur recalculée, qui est la seule cohérente avec le journal.
- **Contrôle de non-négativité historique :** la reconstruction vérifie qu'aucun état intermédiaire n'est négatif. Un écart révèle un antidatage abusif, à corriger par mouvement d'ajustement.
- **Fréquence :** quotidienne, par entreprise, hors heures ouvrées ; `recomputed_at` atteste du passage.

6. Architecture analytique

Principe directeur inchangé depuis la V1 : **aucune logique ML dans les tables transactionnelles**. La V1.1 ajoute la contrepartie manquante — les résultats sont persistés, donc le chemin de lecture utilisateur ne touche jamais un modèle.

```
donnees transactionnelles    (sales, sale_items, sale_returns, inventory_movements,
      |                       expenses, debts, payments, product_prices)
      |  job de reconstruction par plage de jours reaffectes
      v
daily_sales_facts            table d'agregats, RLS, reconstruisible
      |
      |  pipeline de features (anti-fuite temporelle : occurred_at <= as_of)
      v
ml_feature_snapshots         features + labels figes par as_of
      |
      |  entrainement hors ligne, versionne
      v
modele (artefact hors base)
      |
      |  inference batch nocturne
      v
forecasts  /  insights       resultats persistes, RLS
      |
      |  lecture simple, indexee, sans calcul
      v
API FastAPI  ->  UI
```

Couche	Tables	Écrit par	Rôle et règles
Transactionnelle	sales, sale_items, sale_returns, sale_return_items, inventory_movements, expenses, debts, payments, product_prices	API	Vérité immuable et normalisée. Aucune colonne de score, de prédiction ou de feature. Rien n'est supprimé physiquement : la profondeur d'historique détermine la qualité des prévisions — viser 18 à 24 mois avant d'espérer capter une saisonnalité annuelle.
Analytique	**daily_sales_facts** (table)	Job analytique	Série jour × produit, reconstruite par plage. Sert les tableaux de bord et l'exploration. Métriques définies sans ambiguïté en 4.10, retours pré-agrégés avant jointure.
Features ML	ml_feature_snapshots	Pipeline	Matérialise, pour chaque as_of, l'information réellement disponible à cette date. Jamais lue par l'application.
Résultats	**forecasts**, **insights**	Pipeline	Seules tables analytiques exposées à l'API. Écriture par ON CONFLICT DO UPDATE sur la clé d'idempotence. L'application n'écrit que insights.status.
Notification	notifications	Job + API	Dérivée d'un insight critical ou d'une règle de seuil. Jetable.

6.1 Reconstruction de daily_sales_facts — pré-agrégation des retours

La V1 joignait `sale_items` à `sale_return_items` directement. Une ligne de vente ayant reçu trois retours partiels produisait alors trois lignes, donc **trois fois** la quantité vendue, le CA, la remise, le COGS, et trois fois le même `sale_id` dans le comptage. La correction est structurelle : les retours sont agrégés **par `sale_item_id`** dans une CTE, ce qui garantit une ligne par ligne de vente avant toute jointure.

```
WITH cible AS (           -- jours de vente a reconstruire, deduits de l'outbox
  SELECT DISTINCT (s.sold_at AT TIME ZONE c.timezone)::date AS day
    FROM business_events e
    JOIN sales s ON s.company_id = e.company_id AND s.id = e.aggregate_id
    JOIN companies c ON c.id = e.company_id
   WHERE e.company_id = :co AND e.aggregate_type = 'sale'
     AND e.occurred_at > :depuis
),
retours AS (              -- PRE-AGREGATION : une seule ligne par sale_item_id
  SELECT ri.company_id, ri.sale_item_id,
         SUM(ri.qty)                           AS returned_qty,
         SUM(ri.refund_amount - ri.tax_refund)  AS refund_net_amount,
         SUM(ri.qty * ri.unit_cost)             AS cogs_returned
    FROM sale_return_items ri
    JOIN sale_returns r
      ON r.company_id = ri.company_id AND r.id = ri.return_id
   WHERE ri.company_id = :co
   GROUP BY ri.company_id, ri.sale_item_id
),
lignes AS (
  SELECT s.company_id,
         (s.sold_at AT TIME ZONE c.timezone)::date AS day,
         si.product_id, p.category_id, s.id AS sale_id,
         si.qty, si.unit_price, si.discount, si.tax_amount, si.unit_cost,
         COALESCE(rt.returned_qty, 0)       AS returned_qty,
         COALESCE(rt.refund_net_amount, 0)  AS refund_net_amount,
         COALESCE(rt.cogs_returned, 0)      AS cogs_returned
    FROM sales s
    JOIN companies c  ON c.id = s.company_id
    JOIN sale_items si ON si.company_id = s.company_id AND si.sale_id = s.id
    JOIN products p    ON p.company_id = si.company_id AND p.id = si.product_id
    LEFT JOIN retours rt
      ON rt.company_id = si.company_id AND rt.sale_item_id = si.id   -- 1:0..1
   WHERE s.company_id = :co
     AND s.status = 'confirmed'                -- annulees exclues des faits
     AND (s.sold_at AT TIME ZONE c.timezone)::date IN (SELECT day FROM cible)
)
INSERT INTO daily_sales_facts AS f (
  company_id, day, product_id, category_id,
  qty_sold, returned_qty, net_qty_sold,
  gross_revenue, discount_total, tax_total, refund_net_amount, net_revenue,
  cogs_gross, cogs_returned, cogs_net, sale_count, built_at)
SELECT company_id, day, product_id, max(category_id),
       SUM(qty)                                          AS qty_sold,
       SUM(returned_qty)                                 AS returned_qty,
       SUM(qty) - SUM(returned_qty)                       AS net_qty_sold,
       SUM(qty * unit_price)                              AS gross_revenue,
       SUM(discount)                                      AS discount_total,
       SUM(tax_amount)                                    AS tax_total,
       SUM(refund_net_amount)                             AS refund_net_amount,
       SUM(qty * unit_price) - SUM(discount)
         - SUM(refund_net_amount)                         AS net_revenue,
       SUM(qty * unit_cost)                               AS cogs_gross,
       SUM(cogs_returned)                                 AS cogs_returned,
       SUM(qty * unit_cost) - SUM(cogs_returned)          AS cogs_net,
       COUNT(DISTINCT sale_id)                            AS sale_count,
       now()
  FROM lignes
 GROUP BY company_id, day, product_id
ON CONFLICT (company_id, day, product_id) DO UPDATE SET
  category_id = EXCLUDED.category_id, qty_sold = EXCLUDED.qty_sold,
```

```
  returned_qty = EXCLUDED.returned_qty, net_qty_sold = EXCLUDED.net_qty_sold,
  gross_revenue = EXCLUDED.gross_revenue, discount_total = EXCLUDED.discount_total,
  tax_total = EXCLUDED.tax_total, refund_net_amount = EXCLUDED.refund_net_amount,
  net_revenue = EXCLUDED.net_revenue, cogs_gross = EXCLUDED.cogs_gross,
  cogs_returned = EXCLUDED.cogs_returned, cogs_net = EXCLUDED.cogs_net,
  sale_count = EXCLUDED.sale_count, built_at = now();

-- Les lignes devenues vides (vente annulee, derniere ligne supprimee) sont
-- purgees pour les jours reconstruits, sinon un fait fantome subsiste.
DELETE FROM daily_sales_facts d
 WHERE d.company_id = :co
   AND d.day IN (SELECT day FROM cible)
   AND NOT EXISTS (SELECT 1 FROM lignes l
                   WHERE l.company_id = d.company_id AND l.day = d.day
                     AND l.product_id = d.product_id);
```

Propriétés obtenues : une ligne de vente contribue exactement une fois à chaque mesure, quel que soit son nombre de retours ; `COUNT(DISTINCT sale_id)` compte des tickets et non des lignes ; la reconstruction est idempotente (rejouer la même plage donne le même résultat) ; les ventes annulées n'entrent dans aucun fait, et leur annulation purge les faits antérieurs du même jour.

6.2 Anti-fuite temporelle

Une feature calculée pour `as_of = D` ne doit utiliser que des lignes dont `occurred_at <= D`. Comme `daily_sales_facts` est *réécrite* quand un retour arrive plus tard, la relire pour reconstituer l'information d'une date passée introduirait une fuite : on utiliserait un chiffre qui n'était pas connu à cette date. D'où `ml_feature_snapshots`, écrite une fois pour chaque `as_of` et jamais réécrite. La densité de la série reste obligatoire : les jours sans vente valent zéro, sans quoi toute moyenne mobile est biaisée à la hausse. Le signal de rupture (`qty_on_hand` reconstitué à la date, et non aujourd'hui) distingue une demande faible d'une indisponibilité — erreur classique qui fait chuter durablement la prévision d'un article à succès.

7. Isolation multi-tenant

Quatre barrières superposées, dont trois sont structurelles : un bug applicatif ne peut pas les franchir. La V1.1 complète la couverture — plus aucune table tenant-scoped, y compris analytique et journal d'audit, n'échappe au dispositif.

#	Barrière	Portée V1.1
1	company_id NOT NULL + UNIQUE (company_id, id)	Toutes les tables métier, analytiques et de résultats ML. La cible composite est ce qui rend les FK tenant-aware possibles.
2	FK composites (company_id, child_id) → (company_id, parent_id)	Toutes les relations traversant des données tenant-scoped, **y compris les nouvelles** : sale_return_refunds → sale_returns / payments, payments → sale_returns, inventory_movements → inventory_movements, forecasts → products, daily_sales_facts → products.
3	company_id en **première colonne** de tout index composé	La clause RLS est présente dans 100 % des requêtes ; un index qui ne commence pas par company_id est inutilisable.
4	RLS + FORCE ROW LEVEL SECURITY	Toutes les tables ci-dessus, plus audit_logs (correction V1.1), daily_sales_facts, ml_feature_snapshots, forecasts, insights, sale_return_refunds. Le rôle applicatif n'est ni superuser ni BYPASSRLS.

```
-- Politique unique, identique sur toutes les tables tenant-scoped
ALTER TABLE <table> ENABLE ROW LEVEL SECURITY;
ALTER TABLE <table> FORCE  ROW LEVEL SECURITY;   -- s'applique au proprietaire
CREATE POLICY <table>_tenant ON <table>
  USING      (company_id = current_setting('app.company_id', true)::uuid)
  WITH CHECK (company_id = current_setting('app.company_id', true)::uuid);

-- Role applicatif : aucun privilege de contournement
CREATE ROLE coges_app LOGIN NOBYPASSRLS NOSUPERUSER;
GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA public TO coges_app;
REVOKE DELETE ON ALL TABLES IN SCHEMA public FROM coges_app;  -- soft delete only

-- Ouverture de session : pose obligatoire du contexte, portee = transaction
BEGIN;
SET LOCAL app.company_id = '...';   -- issu du JWT, jamais d'un parametre client
-- ... requetes ...
COMMIT;                             -- le contexte disparait avec la transaction
```

- **`SET LOCAL` et non `SET` :** avec un pool de connexions, un SET de session survivrait au retour de la connexion au pool et fuiterait vers la requête suivante, d'un autre tenant. C'est le seul point où une erreur d'implémentation reste possible — d'où le test d'intégrité dédié en section 10.
- **Le `true` de `current_setting` :** évite une erreur si le paramètre est absent. La politique renvoie alors NULL, donc faux, donc **aucune ligne** : l'oubli du contexte produit un jeu vide, jamais une fuite.
- **Écritures du pipeline analytique :** il pose le même SET LOCAL par entreprise et boucle ; il n'utilise pas de rôle privilégié. Une reconstruction ne peut donc pas écrire dans le tenant voisin.
- **`audit_logs` partitionné :** la politique est déclarée sur la table parente et héritée par les partitions ; chaque nouvelle partition mensuelle est créée par la même fonction, ce qui évite une partition sans politique.
- **`companies` et `permissions` :** `companies` reçoit une politique sur `id` (anti-énumération) ; `permissions` est un catalogue global en lecture seule, sans donnée d'entreprise.

8. SQL PostgreSQL 16 complet

Schéma complet V1.1, pas seulement les changements. Exige PostgreSQL 15+ pour `UNIQUE NULLS NOT DISTINCT` ; testé sur 16. Ordre d'exécution : extensions, types, fonctions utilitaires, tables, index, triggers, RLS, couche analytique. Idempotent là où PostgreSQL le permet.

8.1 Extensions, types, fonctions utilitaires

```
-- ======================================================================
-- CoGes V1.1 - schema relationnel complet | PostgreSQL 16 | UTF8 | UTC
-- ======================================================================
BEGIN;

CREATE EXTENSION IF NOT EXISTS pgcrypto;    -- gen_random_uuid()
CREATE EXTENSION IF NOT EXISTS citext;      -- emails, slugs
CREATE EXTENSION IF NOT EXISTS btree_gist;  -- EXCLUDE sur intervalles
CREATE EXTENSION IF NOT EXISTS pg_trgm;     -- recherche par nom

-- ---------------------------------------------------------------- types
CREATE TYPE sale_status    AS ENUM ('draft','confirmed','cancelled');
CREATE TYPE movement_type  AS ENUM ('initial','purchase_in','sale_out',
  'customer_return_in','supplier_return_out','adjustment','loss');
CREATE TYPE expense_kind   AS ENUM ('fixed','variable');
CREATE TYPE party_type     AS ENUM ('customer','supplier');
CREATE TYPE debt_status    AS ENUM ('open','partially_paid','paid','written_off');
CREATE TYPE cash_direction AS ENUM ('inflow','outflow');
CREATE TYPE pay_method     AS ENUM ('cash','mobile_money','bank_transfer',
  'card','cheque');
-- V1.1
CREATE TYPE return_refund_status AS ENUM ('none_due','pending','partial','settled');
CREATE TYPE forecast_type   AS ENUM ('demand_qty','revenue','stockout_date',
  'cash_balance');
CREATE TYPE insight_type    AS ENUM ('stockout_risk','slow_mover','demand_spike',
  'margin_drop','expense_anomaly','cash_shortfall','debt_overdue',
  'cashier_anomaly');
CREATE TYPE insight_severity AS ENUM ('info','warning','critical');
CREATE TYPE insight_status   AS ENUM ('new','read','acknowledged','dismissed',
  'resolved');

-- ------------------------------------------------------ fonctions communes
CREATE OR REPLACE FUNCTION set_updated_at() RETURNS trigger AS $$
BEGIN NEW.updated_at := now(); RETURN NEW; END $$ LANGUAGE plpgsql;

-- Formule CMP unique, partagee par le trigger et la reconstruction :
-- aucune divergence possible entre temps reel et recalcul.
CREATE OR REPLACE FUNCTION cmp_next_avg(
  p_qty numeric, p_avg numeric, p_delta numeric, p_cost numeric)
RETURNS numeric LANGUAGE sql IMMUTABLE AS $$
  SELECT CASE
    WHEN p_delta <= 0            THEN p_avg                -- sortie : inchange
    WHEN p_qty + p_delta = 0     THEN p_avg                -- garde-fou 0/0
    ELSE round((p_qty * p_avg + p_delta * p_cost) / (p_qty + p_delta), 4)
  END;
$$;
```

8.2 Tenant et RBAC

```
CREATE TABLE companies (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
  name        text NOT NULL CHECK (length(btrim(name)) BETWEEN 1 AND 200),
  slug        citext NOT NULL UNIQUE CHECK (slug ~ '^[a-z0-9-]{3,40}$'),
  currency    char(3) NOT NULL DEFAULT 'XOF' CHECK (currency ~ '^[A-Z]{3}$'),
  timezone    text NOT NULL DEFAULT 'UTC',
  settings    jsonb NOT NULL DEFAULT '{}'::jsonb,
  created_at  timestamptz NOT NULL DEFAULT now(),
  updated_at  timestamptz NOT NULL DEFAULT now(),
  deleted_at  timestamptz
);

CREATE TABLE users (
  id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id    uuid NOT NULL REFERENCES companies(id) ON DELETE RESTRICT,
  email         citext NOT NULL CHECK (email ~ '^[^@[:space:]]+@[^@[:space:]]+[.]
[^@[:space:]]+$'),
  password_hash text NOT NULL,
  full_name     text NOT NULL CHECK (length(btrim(full_name)) > 0),
  is_active     boolean NOT NULL DEFAULT true,
  last_login_at timestamptz,
  created_at    timestamptz NOT NULL DEFAULT now(),
  updated_at    timestamptz NOT NULL DEFAULT now(),
  deleted_at    timestamptz,
  CONSTRAINT users_tenant_uq UNIQUE (company_id, id),
  CONSTRAINT users_email_uq  UNIQUE (company_id, email)   -- unicite PAR tenant
);

CREATE TABLE roles (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id  uuid NOT NULL REFERENCES companies(id) ON DELETE RESTRICT,
  name        text NOT NULL CHECK (length(btrim(name)) > 0),
  description text,
  is_system   boolean NOT NULL DEFAULT false,
  created_at  timestamptz NOT NULL DEFAULT now(),
  updated_at  timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT roles_tenant_uq UNIQUE (company_id, id),
  CONSTRAINT roles_name_uq   UNIQUE (company_id, name)
);

-- Catalogue GLOBAL : vocabulaire des droits commun a tous les tenants
CREATE TABLE permissions (
  id       uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  code     text NOT NULL UNIQUE,
  resource text NOT NULL,
  action   text NOT NULL CHECK (action IN ('read','create','update','delete',
                                           'confirm','cancel','export'))
);

CREATE TABLE user_roles (
  user_id    uuid NOT NULL,
  role_id    uuid NOT NULL,
  company_id uuid NOT NULL REFERENCES companies(id) ON DELETE RESTRICT,
  granted_at timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (user_id, role_id),
  FOREIGN KEY (company_id, user_id) REFERENCES users (company_id, id)
    ON DELETE CASCADE,
  FOREIGN KEY (company_id, role_id) REFERENCES roles (company_id, id)
    ON DELETE RESTRICT
);

CREATE TABLE role_permissions (
  role_id       uuid NOT NULL,
```

```
  permission_id uuid NOT NULL REFERENCES permissions(id) ON DELETE RESTRICT,
  company_id    uuid NOT NULL REFERENCES companies(id) ON DELETE RESTRICT,
  PRIMARY KEY (role_id, permission_id),
  FOREIGN KEY (company_id, role_id) REFERENCES roles (company_id, id)
    ON DELETE CASCADE
);
```

8.3 Référentiels — tiers, catégories, produits, prix

```
CREATE TABLE customers (
  id           uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id   uuid NOT NULL REFERENCES companies(id) ON DELETE RESTRICT,
  code         text,
  name         text NOT NULL CHECK (length(btrim(name)) > 0),
  phone        text,
  email        citext,
  credit_limit numeric(14,2) CHECK (credit_limit >= 0),
  created_at   timestamptz NOT NULL DEFAULT now(),
  updated_at   timestamptz NOT NULL DEFAULT now(),
  deleted_at   timestamptz,
  CONSTRAINT customers_tenant_uq UNIQUE (company_id, id)
);

CREATE TABLE suppliers (
  id                 uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id         uuid NOT NULL REFERENCES companies(id) ON DELETE RESTRICT,
  code               text,
  name               text NOT NULL CHECK (length(btrim(name)) > 0),
  phone              text,
  payment_terms_days smallint CHECK (payment_terms_days BETWEEN 0 AND 365),
  created_at         timestamptz NOT NULL DEFAULT now(),
  updated_at         timestamptz NOT NULL DEFAULT now(),
  deleted_at         timestamptz,
  CONSTRAINT suppliers_tenant_uq UNIQUE (company_id, id)
);

CREATE TABLE categories (
  id         uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id uuid NOT NULL REFERENCES companies(id) ON DELETE RESTRICT,
  parent_id  uuid,
  name       text NOT NULL CHECK (length(btrim(name)) > 0),
  created_at timestamptz NOT NULL DEFAULT now(),
  updated_at timestamptz NOT NULL DEFAULT now(),
  deleted_at timestamptz,
  CONSTRAINT categories_tenant_uq UNIQUE (company_id, id),
  CONSTRAINT categories_self_fk FOREIGN KEY (company_id, parent_id)
    REFERENCES categories (company_id, id) ON DELETE RESTRICT,
  CONSTRAINT categories_not_self CHECK (parent_id IS DISTINCT FROM id)
);
CREATE UNIQUE INDEX categories_name_uq
  ON categories (company_id, COALESCE(parent_id, '00000000-0000-0000-0000-000000000000'::uuid),
name)
  WHERE deleted_at IS NULL;

CREATE TABLE products (
  id               uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id       uuid NOT NULL REFERENCES companies(id) ON DELETE RESTRICT,
  category_id      uuid,
  sku              text NOT NULL,
  barcode          text,
```

```
  name             text NOT NULL CHECK (length(btrim(name)) > 0),
  unit             text NOT NULL DEFAULT 'piece'
                     CHECK (unit IN ('piece','kg','litre','carton','metre','paquet')),
  is_stock_tracked boolean NOT NULL DEFAULT true,
  -- V1.1 : parametre de saisie, jamais source de recalcul d'une vente passee
  default_tax_rate numeric(6,4) NOT NULL DEFAULT 0
                     CHECK (default_tax_rate >= 0 AND default_tax_rate < 1),
  reorder_point    numeric(14,3) CHECK (reorder_point >= 0),
  is_active        boolean NOT NULL DEFAULT true,
  created_at       timestamptz NOT NULL DEFAULT now(),
  updated_at       timestamptz NOT NULL DEFAULT now(),
  deleted_at       timestamptz,
  CONSTRAINT products_tenant_uq UNIQUE (company_id, id),
  CONSTRAINT products_sku_uq    UNIQUE (company_id, sku),
  CONSTRAINT products_category_fk FOREIGN KEY (company_id, category_id)
    REFERENCES categories (company_id, id) ON DELETE RESTRICT
);
-- POINT D'EXTENSION DOCUMENTE (non implemente) : une table product_variants
-- viendrait ici, et une colonne variant_id nullable serait ajoutee a
-- sale_items, inventory_movements, product_stock et product_prices.

CREATE TABLE product_prices (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id  uuid NOT NULL REFERENCES companies(id) ON DELETE RESTRICT,
  product_id  uuid NOT NULL,
  unit_price  numeric(14,2) NOT NULL CHECK (unit_price > 0),
  cost_price  numeric(14,4) CHECK (cost_price >= 0),
  valid_from  timestamptz NOT NULL DEFAULT now(),
  valid_to    timestamptz,
  created_by  uuid,
  created_at  timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT product_prices_tenant_uq UNIQUE (company_id, id),
  CONSTRAINT product_prices_product_fk FOREIGN KEY (company_id, product_id)
    REFERENCES products (company_id, id) ON DELETE RESTRICT,
  CONSTRAINT product_prices_creator_fk FOREIGN KEY (company_id, created_by)
    REFERENCES users (company_id, id) ON DELETE RESTRICT,
  CONSTRAINT product_prices_range CHECK (valid_to IS NULL OR valid_to > valid_from),
  -- un seul prix valide a la fois : l'ambiguite tarifaire est impossible
  CONSTRAINT product_prices_no_overlap EXCLUDE USING gist (
    product_id WITH =, tstzrange(valid_from, valid_to) WITH &&)
);

CREATE TABLE product_stock (      -- PROJECTION : aucune verite propre
  product_id       uuid PRIMARY KEY,
  company_id       uuid NOT NULL REFERENCES companies(id) ON DELETE RESTRICT,
  qty_on_hand      numeric(14,3) NOT NULL DEFAULT 0
                     CHECK (qty_on_hand >= 0),          -- stock negatif interdit
  avg_cost         numeric(14,4) NOT NULL DEFAULT 0
                     CHECK (avg_cost >= 0),             -- V1.1 : CMP courant
  stock_value      numeric(14,2) NOT NULL
                     GENERATED ALWAYS AS (round(qty_on_hand * avg_cost, 2)) STORED,
  last_movement_at timestamptz,
  recomputed_at    timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT product_stock_tenant_uq UNIQUE (company_id, product_id),
  CONSTRAINT product_stock_product_fk FOREIGN KEY (company_id, product_id)
    REFERENCES products (company_id, id) ON DELETE RESTRICT
);
```

8.4 Ventes, retours, remboursements

```
CREATE TABLE sales (
  id             uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id     uuid NOT NULL REFERENCES companies(id) ON DELETE RESTRICT,
  customer_id    uuid,
  sale_no        text NOT NULL,
  status         sale_status NOT NULL DEFAULT 'draft',
  sold_at        timestamptz NOT NULL DEFAULT now(),
  subtotal       numeric(14,2) NOT NULL DEFAULT 0 CHECK (subtotal >= 0),
  discount_total numeric(14,2) NOT NULL DEFAULT 0,
  tax_total      numeric(14,2) NOT NULL DEFAULT 0 CHECK (tax_total >= 0), -- V1.1
  -- V1.1 : l'egalite comptable est PRODUITE par la base, pas verifiee par le code
  total          numeric(14,2) NOT NULL
                   GENERATED ALWAYS AS (subtotal - discount_total + tax_total) STORED,
  paid_total     numeric(14,2) NOT NULL DEFAULT 0 CHECK (paid_total >= 0),
  cancel_reason  text,
  confirmed_at   timestamptz,
  cancelled_at   timestamptz,
  created_by     uuid NOT NULL,
  created_at     timestamptz NOT NULL DEFAULT now(),
  updated_at     timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT sales_tenant_uq UNIQUE (company_id, id),
  CONSTRAINT sales_no_uq     UNIQUE (company_id, sale_no),
  CONSTRAINT sales_customer_fk FOREIGN KEY (company_id, customer_id)
    REFERENCES customers (company_id, id) ON DELETE RESTRICT,
  CONSTRAINT sales_creator_fk FOREIGN KEY (company_id, created_by)
    REFERENCES users (company_id, id) ON DELETE RESTRICT,
  CONSTRAINT sales_discount_bounds CHECK (discount_total >= 0
                                      AND discount_total <= subtotal),
  CONSTRAINT sales_cancel_coherent CHECK (
    (status = 'cancelled') = (cancelled_at IS NOT NULL)
    AND (status <> 'cancelled' OR length(btrim(coalesce(cancel_reason,''))) > 0)),
  CONSTRAINT sales_confirm_coherent CHECK (
    status = 'draft' OR confirmed_at IS NOT NULL)
);

CREATE TABLE sale_items (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id  uuid NOT NULL REFERENCES companies(id) ON DELETE RESTRICT,
  sale_id     uuid NOT NULL,
  product_id  uuid NOT NULL,
  price_id    uuid,
  qty         numeric(14,3) NOT NULL CHECK (qty > 0),
  unit_price  numeric(14,2) NOT NULL CHECK (unit_price >= 0),   -- snapshot HT
  unit_cost   numeric(14,4) NOT NULL DEFAULT 0 CHECK (unit_cost >= 0), -- CMP figé
  discount    numeric(14,2) NOT NULL DEFAULT 0 CHECK (discount >= 0),
  tax_rate    numeric(6,4)  NOT NULL DEFAULT 0                  -- V1.1 snapshot
                CHECK (tax_rate >= 0 AND tax_rate < 1),
  net_amount  numeric(14,2) NOT NULL
                GENERATED ALWAYS AS (round(qty * unit_price - discount, 2)) STORED,
  tax_amount  numeric(14,2) NOT NULL
                GENERATED ALWAYS AS (round((qty * unit_price - discount) * tax_rate, 2)) STORED,
  line_total  numeric(14,2) NOT NULL
                GENERATED ALWAYS AS (round(qty * unit_price - discount, 2)
                  + round((qty * unit_price - discount) * tax_rate, 2)) STORED,
  created_at  timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT sale_items_tenant_uq UNIQUE (company_id, id),
  CONSTRAINT sale_items_sale_fk FOREIGN KEY (company_id, sale_id)
    REFERENCES sales (company_id, id) ON DELETE CASCADE,
  CONSTRAINT sale_items_product_fk FOREIGN KEY (company_id, product_id)
```

```
    REFERENCES products (company_id, id) ON DELETE RESTRICT,
  CONSTRAINT sale_items_price_fk FOREIGN KEY (company_id, price_id)
    REFERENCES product_prices (company_id, id) ON DELETE RESTRICT,
  CONSTRAINT sale_items_discount_bounds CHECK (discount <= qty * unit_price)
);

CREATE TABLE sale_returns (
  id             uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id     uuid NOT NULL REFERENCES companies(id) ON DELETE RESTRICT,
  sale_id        uuid NOT NULL,
  return_no      text NOT NULL,
  reason         text,
  returned_at    timestamptz NOT NULL DEFAULT now(),
  total_refund   numeric(14,2) NOT NULL DEFAULT 0 CHECK (total_refund >= 0),
  refunded_total numeric(14,2) NOT NULL DEFAULT 0                      -- V1.1
                   CHECK (refunded_total >= 0),
  refund_status  return_refund_status NOT NULL DEFAULT 'pending',       -- V1.1
  created_by     uuid NOT NULL,
  created_at     timestamptz NOT NULL DEFAULT now(),
  updated_at     timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT sale_returns_tenant_uq UNIQUE (company_id, id),
  CONSTRAINT sale_returns_no_uq     UNIQUE (company_id, return_no),
  CONSTRAINT sale_returns_sale_fk FOREIGN KEY (company_id, sale_id)
    REFERENCES sales (company_id, id) ON DELETE RESTRICT,
  CONSTRAINT sale_returns_creator_fk FOREIGN KEY (company_id, created_by)
    REFERENCES users (company_id, id) ON DELETE RESTRICT,
  -- V1.1 : le double remboursement est structurellement impossible
  CONSTRAINT sale_returns_refund_cap CHECK (refunded_total <= total_refund)
);

CREATE TABLE sale_return_items (
  id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id    uuid NOT NULL REFERENCES companies(id) ON DELETE RESTRICT,
  return_id     uuid NOT NULL,
  sale_item_id  uuid NOT NULL,
  qty           numeric(14,3) NOT NULL CHECK (qty > 0),
  refund_amount numeric(14,2) NOT NULL CHECK (refund_amount >= 0),  -- TTC
  tax_refund    numeric(14,2) NOT NULL DEFAULT 0                    -- V1.1
                  CHECK (tax_refund >= 0),
  unit_cost     numeric(14,4) NOT NULL CHECK (unit_cost >= 0),      -- V1.1
  created_at    timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT sale_return_items_tenant_uq UNIQUE (company_id, id),
  CONSTRAINT sale_return_items_return_fk FOREIGN KEY (company_id, return_id)
    REFERENCES sale_returns (company_id, id) ON DELETE CASCADE,
  CONSTRAINT sale_return_items_item_fk FOREIGN KEY (company_id, sale_item_id)
    REFERENCES sale_items (company_id, id) ON DELETE RESTRICT,
  CONSTRAINT sale_return_items_tax_bounds CHECK (tax_refund <= refund_amount)
);

-- sale_return_refunds depend de payments : voir 8.6
```

8.5 Stock — journal, mouvements inverses, CMP

```
CREATE TABLE inventory_movements (          -- APPEND-ONLY : source de verite
  id                    uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id            uuid NOT NULL REFERENCES companies(id) ON DELETE RESTRICT,
  product_id            uuid NOT NULL,
  type                  movement_type NOT NULL,
  qty_delta             numeric(14,3) NOT NULL CHECK (qty_delta <> 0),
  unit_cost             numeric(14,4) NOT NULL CHECK (unit_cost >= 0),
```

```
  ref_type                text CHECK (ref_type IN ('sale_item','sale_return_item',
                                                   'manual','import','initial')),
  ref_id                  uuid,
  reversal_of_movement_id uuid,                                    -- V1.1
  supplier_id             uuid,
  note                    text,
  occurred_at             timestamptz NOT NULL DEFAULT now(),
  created_by              uuid,
  created_at              timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT inv_tenant_uq UNIQUE (company_id, id),
  CONSTRAINT inv_product_fk FOREIGN KEY (company_id, product_id)
    REFERENCES products (company_id, id) ON DELETE RESTRICT,
  CONSTRAINT inv_supplier_fk FOREIGN KEY (company_id, supplier_id)
    REFERENCES suppliers (company_id, id) ON DELETE RESTRICT,
  CONSTRAINT inv_creator_fk FOREIGN KEY (company_id, created_by)
    REFERENCES users (company_id, id) ON DELETE RESTRICT,
  -- V1.1 : FK auto-referente TENANT-AWARE vers le mouvement annule
  CONSTRAINT inv_reversal_fk FOREIGN KEY (company_id, reversal_of_movement_id)
    REFERENCES inventory_movements (company_id, id) ON DELETE RESTRICT,
  CONSTRAINT inv_reversal_not_self CHECK (reversal_of_movement_id IS DISTINCT FROM id),
  -- V1.1 : un mouvement ne peut etre annule QU'UNE SEULE FOIS
  CONSTRAINT inv_reversal_uq UNIQUE (company_id, reversal_of_movement_id),
  CONSTRAINT inv_sign_coherent CHECK (
    (type IN ('initial','purchase_in','customer_return_in') AND qty_delta > 0)
 OR (type IN ('sale_out','supplier_return_out','loss')      AND qty_delta < 0)
 OR (type = 'adjustment')),
  CONSTRAINT inv_ref_pair CHECK (num_nonnulls(ref_type, ref_id) <> 1)
);

-- V1.1 : idempotence des mouvements DIRECTS uniquement. Sans la clause
-- partielle, le mouvement inverse serait rejete comme doublon de son origine.
CREATE UNIQUE INDEX inv_ref_idempotence
  ON inventory_movements (company_id, ref_type, ref_id)
  WHERE ref_id IS NOT NULL AND reversal_of_movement_id IS NULL;

-- ------------------------------------------- immuabilite du journal
-- V1.1 : exception explicite au lieu d'une RULE "DO INSTEAD NOTHING", qui
-- masquait silencieusement un bug applicatif en renvoyant "0 ligne modifiee".
CREATE OR REPLACE FUNCTION inv_immutable() RETURNS trigger AS $$
BEGIN
  RAISE EXCEPTION 'inventory_movements est append-only : % interdit. '
    'Corriger par un mouvement inverse ou un ajustement.', TG_OP
    USING ERRCODE = 'restrict_violation';
END $$ LANGUAGE plpgsql;
CREATE TRIGGER inv_no_update BEFORE UPDATE ON inventory_movements
  FOR EACH ROW EXECUTE FUNCTION inv_immutable();
CREATE TRIGGER inv_no_delete BEFORE DELETE ON inventory_movements
  FOR EACH ROW EXECUTE FUNCTION inv_immutable();

-- ------------------------------------------- validation du mouvement inverse
-- Nommage t10_/t20_ : les triggers BEFORE se declenchent dans l'ordre
-- alphabetique, la validation doit preceder l'application du CMP.
CREATE OR REPLACE FUNCTION inv_check_reversal() RETURNS trigger AS $$
DECLARE o inventory_movements;
BEGIN
  IF NEW.reversal_of_movement_id IS NULL THEN RETURN NEW; END IF;
  SELECT * INTO o FROM inventory_movements
   WHERE company_id = NEW.company_id AND id = NEW.reversal_of_movement_id
   FOR SHARE;
  IF NOT FOUND THEN
    RAISE EXCEPTION 'Mouvement a annuler introuvable dans ce tenant';
```

```
  END IF;
  IF o.reversal_of_movement_id IS NOT NULL THEN
    RAISE EXCEPTION 'On ne peut pas annuler une annulation (mouvement %)', o.id;
  END IF;
  IF o.product_id <> NEW.product_id THEN
    RAISE EXCEPTION 'Le mouvement inverse doit porter sur le meme produit';
  END IF;
  IF NEW.qty_delta <> -o.qty_delta THEN
    RAISE EXCEPTION 'Quantite inverse attendue % , recue %',
      -o.qty_delta, NEW.qty_delta;
  END IF;
  NEW.unit_cost := o.unit_cost;   -- on rend la marchandise a son cout de sortie
  NEW.ref_type  := o.ref_type;    -- tracabilite conservee vers l'objet metier
  NEW.ref_id    := o.ref_id;
  RETURN NEW;
END $$ LANGUAGE plpgsql;
CREATE TRIGGER t10_inv_check_reversal BEFORE INSERT ON inventory_movements
  FOR EACH ROW EXECUTE FUNCTION inv_check_reversal();

-- ------------------------------------------- application : stock + CMP
CREATE OR REPLACE FUNCTION inv_apply_movement() RETURNS trigger AS $$
DECLARE
  v_tracked boolean;
  v_qty numeric(14,3); v_avg numeric(14,4);
  v_new_qty numeric(14,3); v_new_avg numeric(14,4);
BEGIN
  SELECT is_stock_tracked INTO v_tracked FROM products
   WHERE company_id = NEW.company_id AND id = NEW.product_id;
  IF NOT FOUND THEN
    RAISE EXCEPTION 'Produit % inconnu dans ce tenant', NEW.product_id;
  END IF;
  IF NOT v_tracked THEN
    RAISE EXCEPTION 'Produit % non suivi en stock : aucun mouvement autorise',
      NEW.product_id USING ERRCODE = 'restrict_violation';
  END IF;

  INSERT INTO product_stock (product_id, company_id, qty_on_hand, avg_cost)
  VALUES (NEW.product_id, NEW.company_id, 0, 0)
  ON CONFLICT (product_id) DO NOTHING;

  -- verrou de ligne : serialise deux caisses sur le meme article
  SELECT qty_on_hand, avg_cost INTO v_qty, v_avg FROM product_stock
   WHERE product_id = NEW.product_id FOR UPDATE;

  IF NEW.qty_delta < 0 AND NEW.reversal_of_movement_id IS NULL THEN
    -- SORTIE : le cout de sortie est le CMP applicable a cet instant
    NEW.unit_cost := v_avg;
  END IF;

  v_new_qty := v_qty + NEW.qty_delta;
  IF v_new_qty < 0 THEN
    RAISE EXCEPTION
      'Stock negatif interdit : produit %, disponible %, sortie demandee %',
      NEW.product_id, v_qty, abs(NEW.qty_delta)
      USING ERRCODE = 'check_violation';
  END IF;
  IF NEW.qty_delta > 0 AND NEW.unit_cost IS NULL THEN
    RAISE EXCEPTION 'Entree de stock sans cout unitaire : CMP indefinissable';
  END IF;

  v_new_avg := cmp_next_avg(v_qty, v_avg, NEW.qty_delta, NEW.unit_cost);
```

```
  UPDATE product_stock
     SET qty_on_hand      = v_new_qty,
         avg_cost         = v_new_avg,
         last_movement_at = GREATEST(COALESCE(last_movement_at, NEW.occurred_at),
                                     NEW.occurred_at)
   WHERE product_id = NEW.product_id;
  RETURN NEW;
END $$ LANGUAGE plpgsql;
CREATE TRIGGER t20_inv_apply BEFORE INSERT ON inventory_movements
  FOR EACH ROW EXECUTE FUNCTION inv_apply_movement();

-- ----------------------------------------- reconstruction (job nocturne)
-- Rejoue le journal dans l'ordre canonique et applique la MEME formule CMP.
CREATE OR REPLACE FUNCTION rebuild_product_stock(p_company uuid)
RETURNS TABLE (product_id uuid, qty_ecart numeric, avg_ecart numeric)
LANGUAGE plpgsql AS $$
DECLARE m record; v_qty numeric(14,3); v_avg numeric(14,4); v_prod uuid;
BEGIN
  CREATE TEMP TABLE _rebuilt (product_id uuid PRIMARY KEY,
    qty numeric(14,3), avg numeric(14,4)) ON COMMIT DROP;
  v_prod := NULL;
  FOR m IN
    SELECT * FROM inventory_movements
     WHERE company_id = p_company
     ORDER BY product_id, occurred_at, id        -- ordre canonique
  LOOP
    IF v_prod IS DISTINCT FROM m.product_id THEN
      IF v_prod IS NOT NULL THEN
        INSERT INTO _rebuilt VALUES (v_prod, v_qty, v_avg);
      END IF;
      v_prod := m.product_id; v_qty := 0; v_avg := 0;
    END IF;
    v_avg := cmp_next_avg(v_qty, v_avg, m.qty_delta, m.unit_cost);
    v_qty := v_qty + m.qty_delta;
    IF v_qty < 0 THEN
      RAISE WARNING 'Etat intermediaire negatif : produit %, mouvement % '
        '(antidatage probable)', m.product_id, m.id;
    END IF;
  END LOOP;
  IF v_prod IS NOT NULL THEN INSERT INTO _rebuilt VALUES (v_prod, v_qty, v_avg); END IF;

  RETURN QUERY
    SELECT r.product_id, r.qty - s.qty_on_hand, r.avg - s.avg_cost
      FROM _rebuilt r JOIN product_stock s USING (product_id)
     WHERE r.qty <> s.qty_on_hand OR r.avg <> s.avg_cost;

  UPDATE product_stock s
     SET qty_on_hand = r.qty, avg_cost = r.avg, recomputed_at = now()
    FROM _rebuilt r
   WHERE s.product_id = r.product_id AND s.company_id = p_company;
END $$;
```

8.6 Charges, dettes, paiements, remboursements

```
CREATE TABLE expense_categories (
  id         uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id uuid NOT NULL REFERENCES companies(id) ON DELETE RESTRICT,
  name       text NOT NULL CHECK (length(btrim(name)) > 0),
  kind       expense_kind NOT NULL,
```

```
  created_at timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT expense_categories_tenant_uq UNIQUE (company_id, id),
  CONSTRAINT expense_categories_name_uq   UNIQUE (company_id, name)
);

CREATE TABLE expenses (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id  uuid NOT NULL REFERENCES companies(id) ON DELETE RESTRICT,
  category_id uuid NOT NULL,
  supplier_id uuid,
  label       text NOT NULL CHECK (length(btrim(label)) > 0),
  amount      numeric(14,2) NOT NULL CHECK (amount > 0),
  spent_at    date NOT NULL CHECK (spent_at <= current_date + 1),
  created_by  uuid NOT NULL,
  created_at  timestamptz NOT NULL DEFAULT now(),
  updated_at  timestamptz NOT NULL DEFAULT now(),
  deleted_at  timestamptz,
  CONSTRAINT expenses_tenant_uq UNIQUE (company_id, id),
  CONSTRAINT expenses_category_fk FOREIGN KEY (company_id, category_id)
    REFERENCES expense_categories (company_id, id) ON DELETE RESTRICT,
  CONSTRAINT expenses_supplier_fk FOREIGN KEY (company_id, supplier_id)
    REFERENCES suppliers (company_id, id) ON DELETE RESTRICT,
  CONSTRAINT expenses_creator_fk FOREIGN KEY (company_id, created_by)
    REFERENCES users (company_id, id) ON DELETE RESTRICT
);

CREATE TABLE debts (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id  uuid NOT NULL REFERENCES companies(id) ON DELETE RESTRICT,
  party_type  party_type NOT NULL,
  customer_id uuid,
  supplier_id uuid,
  sale_id     uuid,
  expense_id  uuid,
  principal   numeric(14,2) NOT NULL CHECK (principal > 0),
  amount_paid numeric(14,2) NOT NULL DEFAULT 0 CHECK (amount_paid >= 0),
  balance     numeric(14,2) NOT NULL
                GENERATED ALWAYS AS (principal - amount_paid) STORED,
  due_date    date,
  status      debt_status NOT NULL DEFAULT 'open',
  closed_at   timestamptz,
  created_at  timestamptz NOT NULL DEFAULT now(),
  updated_at  timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT debts_tenant_uq UNIQUE (company_id, id),
  CONSTRAINT debts_customer_fk FOREIGN KEY (company_id, customer_id)
    REFERENCES customers (company_id, id) ON DELETE RESTRICT,
  CONSTRAINT debts_supplier_fk FOREIGN KEY (company_id, supplier_id)
    REFERENCES suppliers (company_id, id) ON DELETE RESTRICT,
  CONSTRAINT debts_sale_fk FOREIGN KEY (company_id, sale_id)
    REFERENCES sales (company_id, id) ON DELETE RESTRICT,
  CONSTRAINT debts_expense_fk FOREIGN KEY (company_id, expense_id)
    REFERENCES expenses (company_id, id) ON DELETE RESTRICT,
  CONSTRAINT debts_party_xor CHECK (
    (party_type = 'customer' AND customer_id IS NOT NULL AND supplier_id IS NULL)
 OR (party_type = 'supplier' AND supplier_id IS NOT NULL AND customer_id IS NULL)),
  CONSTRAINT debts_origin_one CHECK (num_nonnulls(sale_id, expense_id) <= 1),
  CONSTRAINT debts_paid_bounds CHECK (amount_paid <= principal),
  CONSTRAINT debts_closed_coherent CHECK (
    (status IN ('paid','written_off')) = (closed_at IS NOT NULL))
);
```

```
CREATE TABLE payments (
  id             uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id     uuid NOT NULL REFERENCES companies(id) ON DELETE RESTRICT,
  direction      cash_direction NOT NULL,
  method         pay_method NOT NULL,
  amount         numeric(14,2) NOT NULL CHECK (amount > 0),
  paid_at        timestamptz NOT NULL DEFAULT now(),
  sale_id        uuid,
  expense_id     uuid,
  sale_return_id uuid,                                          -- V1.1
  reference      text,
  note           text,
  created_by     uuid NOT NULL,
  created_at     timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT payments_tenant_uq UNIQUE (company_id, id),
  CONSTRAINT payments_sale_fk FOREIGN KEY (company_id, sale_id)
    REFERENCES sales (company_id, id) ON DELETE RESTRICT,
  CONSTRAINT payments_expense_fk FOREIGN KEY (company_id, expense_id)
    REFERENCES expenses (company_id, id) ON DELETE RESTRICT,
  CONSTRAINT payments_return_fk FOREIGN KEY (company_id, sale_return_id)
    REFERENCES sale_returns (company_id, id) ON DELETE RESTRICT,
  CONSTRAINT payments_creator_fk FOREIGN KEY (company_id, created_by)
    REFERENCES users (company_id, id) ON DELETE RESTRICT,
  -- V1.1 : au plus un contexte direct, et sens financier coherent
  CONSTRAINT payments_context_one CHECK (
    num_nonnulls(sale_id, expense_id, sale_return_id) <= 1),
  CONSTRAINT payments_return_outflow CHECK (
    sale_return_id IS NULL OR direction = 'outflow')
);

CREATE TABLE payment_allocations (
  id         uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id uuid NOT NULL REFERENCES companies(id) ON DELETE RESTRICT,
  payment_id uuid NOT NULL,
  debt_id    uuid NOT NULL,
  amount     numeric(14,2) NOT NULL CHECK (amount > 0),
  created_at timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT pa_tenant_uq UNIQUE (company_id, id),
  CONSTRAINT pa_pair_uq   UNIQUE (company_id, payment_id, debt_id),
  CONSTRAINT pa_payment_fk FOREIGN KEY (company_id, payment_id)
    REFERENCES payments (company_id, id) ON DELETE CASCADE,
  CONSTRAINT pa_debt_fk FOREIGN KEY (company_id, debt_id)
    REFERENCES debts (company_id, id) ON DELETE RESTRICT
);

-- V1.1 : lettrage retour <-> paiement, symetrique de payment_allocations.
-- Permet un remboursement echelonne, et un versement couvrant deux retours.
CREATE TABLE sale_return_refunds (
  id             uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id     uuid NOT NULL REFERENCES companies(id) ON DELETE RESTRICT,
  sale_return_id uuid NOT NULL,
  payment_id     uuid NOT NULL,
  amount         numeric(14,2) NOT NULL CHECK (amount > 0),
  created_at     timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT srr_tenant_uq UNIQUE (company_id, id),
  CONSTRAINT srr_pair_uq   UNIQUE (company_id, payment_id, sale_return_id),
  CONSTRAINT srr_return_fk FOREIGN KEY (company_id, sale_return_id)
    REFERENCES sale_returns (company_id, id) ON DELETE RESTRICT,
  CONSTRAINT srr_payment_fk FOREIGN KEY (company_id, payment_id)
    REFERENCES payments (company_id, id) ON DELETE CASCADE
);
```

8.7 Triggers financiers et de cohérence des ventes

```
-- ---------------------------------- totaux de vente (subtotal / remise / taxe)
CREATE OR REPLACE FUNCTION sale_recompute_totals() RETURNS trigger AS $$
DECLARE v_sale uuid; v_co uuid;
BEGIN
  v_sale := COALESCE(NEW.sale_id, OLD.sale_id);
  v_co   := COALESCE(NEW.company_id, OLD.company_id);
  UPDATE sales s SET
    subtotal       = COALESCE(t.sub, 0),
    discount_total = COALESCE(t.dis, 0),
    tax_total      = COALESCE(t.tax, 0),
    updated_at     = now()
  FROM (SELECT SUM(qty * unit_price) sub, SUM(discount) dis, SUM(tax_amount) tax
          FROM sale_items
         WHERE company_id = v_co AND sale_id = v_sale) t
  WHERE s.company_id = v_co AND s.id = v_sale;
  RETURN NULL;
END $$ LANGUAGE plpgsql;
CREATE TRIGGER sale_items_totals AFTER INSERT OR UPDATE OR DELETE ON sale_items
  FOR EACH ROW EXECUTE FUNCTION sale_recompute_totals();

-- ---------------------------------- immuabilite des lignes apres confirmation
CREATE OR REPLACE FUNCTION sale_items_frozen() RETURNS trigger AS $$
DECLARE v_status sale_status;
BEGIN
  SELECT status INTO v_status FROM sales
   WHERE company_id = COALESCE(NEW.company_id, OLD.company_id)
     AND id = COALESCE(NEW.sale_id, OLD.sale_id);
  IF v_status <> 'draft' THEN
    RAISE EXCEPTION 'Vente % : les lignes sont figees apres confirmation',
      COALESCE(NEW.sale_id, OLD.sale_id) USING ERRCODE = 'restrict_violation';
  END IF;
  RETURN COALESCE(NEW, OLD);
END $$ LANGUAGE plpgsql;
CREATE TRIGGER sale_items_no_change BEFORE UPDATE OR DELETE ON sale_items
  FOR EACH ROW EXECUTE FUNCTION sale_items_frozen();

-- ---------------------------------- retour : quantite bornee par la vente
CREATE OR REPLACE FUNCTION return_item_check() RETURNS trigger AS $$
DECLARE v_sold numeric(14,3); v_already numeric(14,3);
BEGIN
  SELECT qty INTO v_sold FROM sale_items
   WHERE company_id = NEW.company_id AND id = NEW.sale_item_id FOR UPDATE;
  IF NOT FOUND THEN RAISE EXCEPTION 'Ligne de vente introuvable'; END IF;
  SELECT COALESCE(SUM(qty), 0) INTO v_already FROM sale_return_items
   WHERE company_id = NEW.company_id AND sale_item_id = NEW.sale_item_id
     AND id <> NEW.id;
  IF v_already + NEW.qty > v_sold THEN
    RAISE EXCEPTION 'Retour cumule % superieur a la quantite vendue %',
      v_already + NEW.qty, v_sold USING ERRCODE = 'check_violation';
  END IF;
  RETURN NEW;
END $$ LANGUAGE plpgsql;
CREATE TRIGGER sri_check BEFORE INSERT OR UPDATE ON sale_return_items
  FOR EACH ROW EXECUTE FUNCTION return_item_check();

-- ---------------------------------- total remboursable + statut du retour
CREATE OR REPLACE FUNCTION return_recompute() RETURNS trigger AS $$
DECLARE v_ret uuid; v_co uuid;
BEGIN
```

```
  v_ret := COALESCE(NEW.return_id, OLD.return_id);
  v_co  := COALESCE(NEW.company_id, OLD.company_id);
  UPDATE sale_returns r SET total_refund = COALESCE(t.s, 0), updated_at = now()
    FROM (SELECT SUM(refund_amount) s FROM sale_return_items
          WHERE company_id = v_co AND return_id = v_ret) t
   WHERE r.company_id = v_co AND r.id = v_ret;
  RETURN NULL;
END $$ LANGUAGE plpgsql;
CREATE TRIGGER sri_totals AFTER INSERT OR UPDATE OR DELETE ON sale_return_items
  FOR EACH ROW EXECUTE FUNCTION return_recompute();

CREATE OR REPLACE FUNCTION return_refund_status_set() RETURNS trigger AS $$
BEGIN
  NEW.refund_status := CASE
    WHEN NEW.total_refund = 0                    THEN 'none_due'
    WHEN NEW.refunded_total = 0                  THEN 'pending'
    WHEN NEW.refunded_total < NEW.total_refund   THEN 'partial'
    ELSE 'settled' END::return_refund_status;
  RETURN NEW;
END $$ LANGUAGE plpgsql;
CREATE TRIGGER sr_status BEFORE INSERT OR UPDATE OF total_refund, refunded_total
  ON sale_returns FOR EACH ROW EXECUTE FUNCTION return_refund_status_set();

-- ---------------------------------- remboursement : plafonds et sens financier
CREATE OR REPLACE FUNCTION refund_check() RETURNS trigger AS $$
DECLARE v_due numeric(14,2); v_free numeric(14,2); v_dir cash_direction;
BEGIN
  SELECT total_refund - refunded_total INTO v_due FROM sale_returns
   WHERE company_id = NEW.company_id AND id = NEW.sale_return_id FOR UPDATE;
  IF NOT FOUND THEN RAISE EXCEPTION 'Retour introuvable'; END IF;
  IF NEW.amount > v_due THEN
    RAISE EXCEPTION 'Remboursement % superieur au reste du %', NEW.amount, v_due
      USING ERRCODE = 'check_violation';
  END IF;
  SELECT direction, amount - COALESCE(
           (SELECT SUM(r.amount) FROM sale_return_refunds r
             WHERE r.payment_id = p.id AND r.id <> NEW.id), 0)
    INTO v_dir, v_free
    FROM payments p
   WHERE p.company_id = NEW.company_id AND p.id = NEW.payment_id FOR UPDATE;
  IF v_dir <> 'outflow' THEN
    RAISE EXCEPTION 'Un remboursement de retour exige direction = outflow';
  END IF;
  IF NEW.amount > v_free THEN
    RAISE EXCEPTION 'Paiement sur-affecte : reste % disponible', v_free;
  END IF;
  RETURN NEW;
END $$ LANGUAGE plpgsql;
CREATE TRIGGER srr_check BEFORE INSERT OR UPDATE ON sale_return_refunds
  FOR EACH ROW EXECUTE FUNCTION refund_check();

CREATE OR REPLACE FUNCTION refund_rollup() RETURNS trigger AS $$
DECLARE v_ret uuid; v_co uuid;
BEGIN
  v_ret := COALESCE(NEW.sale_return_id, OLD.sale_return_id);
  v_co  := COALESCE(NEW.company_id, OLD.company_id);
  UPDATE sale_returns r SET refunded_total = COALESCE(t.s, 0), updated_at = now()
    FROM (SELECT SUM(amount) s FROM sale_return_refunds
          WHERE company_id = v_co AND sale_return_id = v_ret) t
   WHERE r.company_id = v_co AND r.id = v_ret;
  RETURN NULL;
```

```
END $$ LANGUAGE plpgsql;
CREATE TRIGGER srr_rollup AFTER INSERT OR UPDATE OR DELETE ON sale_return_refunds
  FOR EACH ROW EXECUTE FUNCTION refund_rollup();

-- ---------------------------------- lettrage paiement / dette (V1, conserve)
CREATE OR REPLACE FUNCTION allocation_check() RETURNS trigger AS $$
DECLARE v_bal numeric(14,2); v_free numeric(14,2);
BEGIN
  SELECT balance INTO v_bal FROM debts
   WHERE company_id = NEW.company_id AND id = NEW.debt_id FOR UPDATE;
  IF NEW.amount > v_bal THEN
    RAISE EXCEPTION 'Affectation % superieure au solde restant %',
      NEW.amount, v_bal USING ERRCODE = 'check_violation';
  END IF;
  SELECT p.amount - COALESCE((SELECT SUM(a.amount) FROM payment_allocations a
                              WHERE a.payment_id = p.id AND a.id <> NEW.id), 0)
    INTO v_free FROM payments p
   WHERE p.company_id = NEW.company_id AND p.id = NEW.payment_id FOR UPDATE;
  IF NEW.amount > v_free THEN
    RAISE EXCEPTION 'Paiement sur-affecte : reste % disponible', v_free;
  END IF;
  RETURN NEW;
END $$ LANGUAGE plpgsql;
CREATE TRIGGER pa_check BEFORE INSERT OR UPDATE ON payment_allocations
  FOR EACH ROW EXECUTE FUNCTION allocation_check();

CREATE OR REPLACE FUNCTION debt_rollup() RETURNS trigger AS $$
DECLARE v_debt uuid; v_co uuid;
BEGIN
  v_debt := COALESCE(NEW.debt_id, OLD.debt_id);
  v_co   := COALESCE(NEW.company_id, OLD.company_id);
  UPDATE debts d SET
    amount_paid = COALESCE(t.s, 0),
    status = CASE WHEN COALESCE(t.s, 0) = 0             THEN 'open'
                  WHEN COALESCE(t.s, 0) < d.principal   THEN 'partially_paid'
                  ELSE 'paid' END::debt_status,
    closed_at = CASE WHEN COALESCE(t.s, 0) >= d.principal THEN now() END,
    updated_at = now()
  FROM (SELECT SUM(amount) s FROM payment_allocations
        WHERE company_id = v_co AND debt_id = v_debt) t
  WHERE d.company_id = v_co AND d.id = v_debt AND d.status <> 'written_off';
  RETURN NULL;
END $$ LANGUAGE plpgsql;
CREATE TRIGGER pa_rollup AFTER INSERT OR UPDATE OR DELETE ON payment_allocations
  FOR EACH ROW EXECUTE FUNCTION debt_rollup();

-- ---------------------------------- plafond de credit client
CREATE OR REPLACE FUNCTION debt_credit_limit() RETURNS trigger AS $$
DECLARE v_limit numeric(14,2); v_open numeric(14,2);
BEGIN
  IF NEW.party_type <> 'customer' THEN RETURN NEW; END IF;
  SELECT credit_limit INTO v_limit FROM customers
   WHERE company_id = NEW.company_id AND id = NEW.customer_id;
  IF v_limit IS NULL THEN RETURN NEW; END IF;
  SELECT COALESCE(SUM(balance), 0) INTO v_open FROM debts
   WHERE company_id = NEW.company_id AND customer_id = NEW.customer_id
     AND status IN ('open','partially_paid') AND id <> NEW.id;
  IF v_open + NEW.principal > v_limit THEN
    RAISE EXCEPTION 'Plafond de credit depasse : % + % > %',
      v_open, NEW.principal, v_limit USING ERRCODE = 'check_violation';
  END IF;
```

```
  RETURN NEW;
END $$ LANGUAGE plpgsql;
CREATE TRIGGER debts_credit_limit BEFORE INSERT ON debts
  FOR EACH ROW EXECUTE FUNCTION debt_credit_limit();

-- ---------------------------------- paid_total de la vente
CREATE OR REPLACE FUNCTION sale_paid_rollup() RETURNS trigger AS $$
DECLARE v_sale uuid; v_co uuid;
BEGIN
  v_sale := COALESCE(NEW.sale_id, OLD.sale_id);
  IF v_sale IS NULL THEN RETURN NULL; END IF;
  v_co := COALESCE(NEW.company_id, OLD.company_id);
  UPDATE sales s SET paid_total = COALESCE(t.s, 0), updated_at = now()
    FROM (SELECT SUM(amount) s FROM payments
           WHERE company_id = v_co AND sale_id = v_sale AND direction = 'inflow') t
   WHERE s.company_id = v_co AND s.id = v_sale;
  RETURN NULL;
END $$ LANGUAGE plpgsql;
CREATE TRIGGER payments_sale_rollup AFTER INSERT OR UPDATE OR DELETE ON payments
  FOR EACH ROW EXECUTE FUNCTION sale_paid_rollup();
```

8.8 Journaux — notifications, audit, outbox

```
CREATE TABLE notifications (
  id         uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id uuid NOT NULL REFERENCES companies(id) ON DELETE RESTRICT,
  user_id    uuid,
  type       text NOT NULL CHECK (type IN ('stock_low','stock_out','debt_due',
               'sale_cancelled','forecast_alert','insight_critical')),
  title      text NOT NULL,
  body       text,
  data       jsonb NOT NULL DEFAULT '{}'::jsonb,
  read_at    timestamptz,
  created_at timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT notifications_tenant_uq UNIQUE (company_id, id),
  CONSTRAINT notifications_user_fk FOREIGN KEY (company_id, user_id)
    REFERENCES users (company_id, id) ON DELETE CASCADE
);

-- Partitionne par mois : purge par DROP PARTITION, pas par DELETE massif
CREATE TABLE audit_logs (
  id            bigint GENERATED ALWAYS AS IDENTITY,
  company_id    uuid NOT NULL,
  actor_user_id uuid,
  action        text NOT NULL CHECK (action IN ('insert','update','delete',
                  'login','export','confirm','cancel','rebuild')),
  entity_table  text NOT NULL,
  entity_id     uuid,
  diff          jsonb,
  ip_address    inet,
  occurred_at   timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (id, occurred_at)
) PARTITION BY RANGE (occurred_at);
CREATE TABLE audit_logs_2026m09 PARTITION OF audit_logs
  FOR VALUES FROM ('2026-09-01') TO ('2026-10-01');
-- Les partitions suivantes sont creees par un job ; la politique RLS declaree
-- sur la table parente est heritee, aucune partition ne peut naitre sans elle.

CREATE TABLE business_events (          -- TRANSACTIONAL OUTBOX
  id             bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  company_id     uuid NOT NULL REFERENCES companies(id) ON DELETE RESTRICT,
  event_type     text NOT NULL,
  aggregate_type text NOT NULL,
  aggregate_id   uuid NOT NULL,
  payload        jsonb NOT NULL,
  occurred_at    timestamptz NOT NULL DEFAULT now(),
  published_at   timestamptz,
  attempts       smallint NOT NULL DEFAULT 0 CHECK (attempts <= 10)
);
```

8.9 Couche analytique et résultats ML

```
-- V1.1 : TABLE et non plus vue materialisee -> soumise a RLS comme le reste
CREATE TABLE daily_sales_facts (
  company_id        uuid NOT NULL REFERENCES companies(id) ON DELETE RESTRICT,
  day               date NOT NULL,
  product_id        uuid NOT NULL,
  category_id       uuid,
  qty_sold          numeric(14,3) NOT NULL DEFAULT 0 CHECK (qty_sold >= 0),
  returned_qty      numeric(14,3) NOT NULL DEFAULT 0 CHECK (returned_qty >= 0),
  net_qty_sold      numeric(14,3) NOT NULL DEFAULT 0,
```

```
  gross_revenue     numeric(14,2) NOT NULL DEFAULT 0,
  discount_total    numeric(14,2) NOT NULL DEFAULT 0,
  tax_total         numeric(14,2) NOT NULL DEFAULT 0,
  refund_net_amount numeric(14,2) NOT NULL DEFAULT 0,
  net_revenue       numeric(14,2) NOT NULL DEFAULT 0,
  cogs_gross        numeric(14,2) NOT NULL DEFAULT 0,
  cogs_returned     numeric(14,2) NOT NULL DEFAULT 0,
  cogs_net          numeric(14,2) NOT NULL DEFAULT 0,
  sale_count        integer NOT NULL DEFAULT 0 CHECK (sale_count >= 0),
  built_at          timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (company_id, day, product_id),
  CONSTRAINT dsf_product_fk FOREIGN KEY (company_id, product_id)
    REFERENCES products (company_id, id) ON DELETE RESTRICT,
  CONSTRAINT dsf_net_qty_coherent CHECK (net_qty_sold = qty_sold - returned_qty),
  CONSTRAINT dsf_returned_bounds  CHECK (returned_qty <= qty_sold)
);

CREATE TABLE ml_feature_snapshots (     -- ecrite par le pipeline uniquement
  id         bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  company_id uuid NOT NULL REFERENCES companies(id) ON DELETE RESTRICT,
  product_id uuid NOT NULL,
  as_of      date NOT NULL,
  features   jsonb NOT NULL,
  labels     jsonb,
  created_at timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT mlf_uq UNIQUE (company_id, product_id, as_of),
  CONSTRAINT mlf_product_fk FOREIGN KEY (company_id, product_id)
    REFERENCES products (company_id, id) ON DELETE RESTRICT
);

-- V1.1 : previsions persistees. L'API lit, elle n'infere jamais.
CREATE TABLE forecasts (
  id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id    uuid NOT NULL REFERENCES companies(id) ON DELETE RESTRICT,
  product_id    uuid,                       -- NULL = prevision entreprise
  forecast_type forecast_type NOT NULL,
  as_of         date NOT NULL,
  horizon_date  date NOT NULL,
  yhat          numeric(14,4) NOT NULL,
  yhat_lower    numeric(14,4),
  yhat_upper    numeric(14,4),
  model_name    text NOT NULL,
  model_version text NOT NULL,
  metadata      jsonb NOT NULL DEFAULT '{}'::jsonb,
  generated_at  timestamptz NOT NULL DEFAULT now(),
  CONSTRAINT forecasts_tenant_uq UNIQUE (company_id, id),
  CONSTRAINT forecasts_product_fk FOREIGN KEY (company_id, product_id)
    REFERENCES products (company_id, id) ON DELETE RESTRICT,
  CONSTRAINT forecasts_horizon CHECK (horizon_date >= as_of),
  CONSTRAINT forecasts_interval CHECK (
    yhat_lower IS NULL OR yhat_upper IS NULL OR yhat_lower <= yhat_upper),
  -- PostgreSQL 15+ : product_id nullable doit participer a l'unicite
  CONSTRAINT forecasts_idem UNIQUE NULLS NOT DISTINCT
    (company_id, forecast_type, product_id, as_of, horizon_date, model_version)
);

-- V1.1 : anomalies, observations, recommandations persistees
CREATE TABLE insights (
  id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id    uuid NOT NULL REFERENCES companies(id) ON DELETE RESTRICT,
  insight_type  insight_type NOT NULL,
```

```
  severity      insight_severity NOT NULL DEFAULT 'info',
  title         text NOT NULL CHECK (length(btrim(title)) BETWEEN 1 AND 160),
  body          text,
  entity_table  text CHECK (entity_table IN ('products','customers','suppliers',
                  'expense_categories','sales','users')),
  entity_id     uuid,
  evidence      jsonb NOT NULL DEFAULT '{}'::jsonb,
  dedupe_key    text NOT NULL,
  model_name    text,
  model_version text,
  status        insight_status NOT NULL DEFAULT 'new',
  read_at       timestamptz,
  resolved_at   timestamptz,
  generated_at  timestamptz NOT NULL DEFAULT now(),
  valid_until   timestamptz,
  CONSTRAINT insights_tenant_uq UNIQUE (company_id, id),
  -- reference souple assumee : pas de FK polymorphe, l'insight survit a
  -- l'archivage de sa cible ; les deux colonnes vont par paire
  CONSTRAINT insights_entity_pair CHECK (num_nonnulls(entity_table, entity_id) <> 1),
  CONSTRAINT insights_read_coherent CHECK (status <> 'new' OR read_at IS NULL),
  CONSTRAINT insights_resolved_coherent CHECK (
    (status = 'resolved') = (resolved_at IS NOT NULL))
);
-- Idempotence : un rejeu met a jour l'insight vivant. Un insight ecarte par
-- l'utilisateur ne bloque pas une reemission si le probleme reapparait.
CREATE UNIQUE INDEX insights_dedupe_uq ON insights (company_id, dedupe_key)
  WHERE status <> 'dismissed';
```

8.10 Index

```
-- company_id est TOUJOURS la premiere colonne : la clause RLS est dans 100 %
-- des requetes, un index qui ne commence pas par company_id est inutilisable.
CREATE UNIQUE INDEX companies_live_uq ON companies (id) WHERE deleted_at IS NULL;
CREATE INDEX users_live       ON users (company_id) WHERE deleted_at IS NULL;
CREATE INDEX user_roles_user  ON user_roles (company_id, user_id);
CREATE INDEX role_perms_role  ON role_permissions (company_id, role_id);

CREATE UNIQUE INDEX customers_code_uq ON customers (company_id, code)
  WHERE code IS NOT NULL AND deleted_at IS NULL;
CREATE UNIQUE INDEX suppliers_code_uq ON suppliers (company_id, code)
  WHERE code IS NOT NULL AND deleted_at IS NULL;
CREATE INDEX customers_name     ON customers (company_id, name);
CREATE INDEX customers_name_trgm ON customers USING gin (name gin_trgm_ops);
CREATE INDEX suppliers_name_trgm ON suppliers USING gin (name gin_trgm_ops);
CREATE INDEX categories_parent  ON categories (company_id, parent_id);

CREATE UNIQUE INDEX products_barcode_uq ON products (company_id, barcode)
  WHERE barcode IS NOT NULL AND deleted_at IS NULL;
CREATE INDEX products_category ON products (company_id, category_id);
CREATE INDEX products_sellable ON products (company_id)
  WHERE is_active AND deleted_at IS NULL;
CREATE INDEX products_name_trgm ON products USING gin (name gin_trgm_ops);

CREATE INDEX prices_history ON product_prices (company_id, product_id, valid_from DESC);
CREATE INDEX prices_current ON product_prices (company_id, product_id)
  WHERE valid_to IS NULL;

CREATE INDEX stock_threshold ON product_stock (company_id, qty_on_hand);
CREATE INDEX stock_out       ON product_stock (company_id) WHERE qty_on_hand = 0;
```

```
CREATE INDEX sales_recent    ON sales (company_id, sold_at DESC);
CREATE INDEX sales_customer  ON sales (company_id, customer_id, sold_at DESC);
CREATE INDEX sales_confirmed ON sales (company_id, sold_at)
  WHERE status = 'confirmed';
CREATE INDEX sales_sold_brin ON sales USING brin (sold_at);
CREATE INDEX sale_items_sale    ON sale_items (company_id, sale_id);
CREATE INDEX sale_items_product ON sale_items (company_id, product_id);

CREATE INDEX returns_sale   ON sale_returns (company_id, sale_id);
CREATE INDEX returns_recent ON sale_returns (company_id, returned_at DESC);
CREATE INDEX returns_unpaid ON sale_returns (company_id, refund_status)
  WHERE refund_status IN ('pending','partial');
CREATE INDEX sri_item      ON sale_return_items (company_id, sale_item_id);
CREATE INDEX srr_by_return ON sale_return_refunds (company_id, sale_return_id);
CREATE INDEX srr_by_payment ON sale_return_refunds (company_id, payment_id);

CREATE INDEX inv_stock_calc ON inventory_movements
  (company_id, product_id, occurred_at DESC);
CREATE INDEX inv_ref_lookup ON inventory_movements (company_id, ref_type, ref_id);
CREATE INDEX inv_occurred_brin ON inventory_movements USING brin (occurred_at);

CREATE INDEX expenses_recent ON expenses (company_id, spent_at DESC)
  WHERE deleted_at IS NULL;
CREATE INDEX expenses_cat    ON expenses (company_id, category_id, spent_at);

CREATE INDEX debts_party  ON debts (company_id, party_type, status);
CREATE INDEX debts_client ON debts (company_id, customer_id)
  WHERE status <> 'paid';
CREATE INDEX debts_due    ON debts (company_id, due_date)
  WHERE status IN ('open','partially_paid');

CREATE INDEX payments_recent ON payments (company_id, paid_at DESC);
CREATE INDEX payments_dir    ON payments (company_id, direction, paid_at);
CREATE INDEX payments_brin   ON payments USING brin (paid_at);
CREATE UNIQUE INDEX payments_ref_uq ON payments (company_id, reference)
  WHERE reference IS NOT NULL;
CREATE INDEX pa_debt    ON payment_allocations (company_id, debt_id);
CREATE INDEX pa_payment ON payment_allocations (company_id, payment_id);

CREATE INDEX notifs_unread ON notifications (company_id, user_id)
  WHERE read_at IS NULL;
CREATE INDEX audit_entity  ON audit_logs (company_id, entity_table, entity_id);
CREATE INDEX audit_actor   ON audit_logs (company_id, actor_user_id, occurred_at DESC);
CREATE INDEX outbox_queue  ON business_events (company_id, occurred_at)
  WHERE published_at IS NULL;

CREATE INDEX dsf_day      ON daily_sales_facts (company_id, day);
CREATE INDEX dsf_product  ON daily_sales_facts (company_id, product_id, day);
CREATE INDEX dsf_category ON daily_sales_facts (company_id, category_id, day);
CREATE INDEX dsf_day_brin ON daily_sales_facts USING brin (day);

CREATE INDEX forecasts_read ON forecasts
  (company_id, product_id, forecast_type, horizon_date);
CREATE INDEX forecasts_asof ON forecasts (company_id, as_of DESC);
CREATE INDEX insights_feed  ON insights
  (company_id, status, severity, generated_at DESC);
CREATE INDEX insights_entity ON insights (company_id, entity_table, entity_id);
```

8.11 Row Level Security — couverture complète

```
-- Boucle explicite sur TOUTES les tables tenant-scoped, y compris celles
-- ajoutees en V1.1 et audit_logs, oublie en V1.
DO $$
DECLARE t text;
BEGIN
  FOREACH t IN ARRAY ARRAY[
    'users','roles','user_roles','role_permissions',
    'customers','suppliers','categories','products','product_prices',
    'product_stock','sales','sale_items','sale_returns','sale_return_items',
    'sale_return_refunds','inventory_movements',
    'expense_categories','expenses','debts','payments','payment_allocations',
    'notifications','audit_logs','business_events',
    'daily_sales_facts','ml_feature_snapshots','forecasts','insights'
  ] LOOP
    EXECUTE format('ALTER TABLE %I ENABLE ROW LEVEL SECURITY', t);
    EXECUTE format('ALTER TABLE %I FORCE  ROW LEVEL SECURITY', t);
    EXECUTE format($f$
      CREATE POLICY %1$I_tenant ON %1$I
        USING      (company_id = current_setting('app.company_id', true)::uuid)
        WITH CHECK (company_id = current_setting('app.company_id', true)::uuid)
    $f$, t);
  END LOOP;
END $$;

-- companies : politique sur id, pour empecher l'enumeration des tenants
ALTER TABLE companies ENABLE ROW LEVEL SECURITY;
ALTER TABLE companies FORCE  ROW LEVEL SECURITY;
CREATE POLICY companies_self ON companies
  USING      (id = current_setting('app.company_id', true)::uuid)
  WITH CHECK (id = current_setting('app.company_id', true)::uuid);
-- permissions : catalogue global, aucune donnee d'entreprise, pas de RLS.

-- Role applicatif : aucun privilege de contournement, aucun DELETE physique
CREATE ROLE coges_app LOGIN NOSUPERUSER NOBYPASSRLS NOCREATEDB NOCREATEROLE;
GRANT USAGE ON SCHEMA public TO coges_app;
GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA public TO coges_app;
GRANT SELECT ON permissions TO coges_app;
GRANT USAGE ON ALL SEQUENCES IN SCHEMA public TO coges_app;

-- Role analytique : ecrit les projections, lit le transactionnel, pose le
-- meme SET LOCAL par entreprise. Pas de BYPASSRLS non plus.
CREATE ROLE coges_analytics LOGIN NOSUPERUSER NOBYPASSRLS;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO coges_analytics;
GRANT INSERT, UPDATE, DELETE ON daily_sales_facts, ml_feature_snapshots,
  forecasts, insights TO coges_analytics;

COMMIT;
```

9. Migration Alembic V1 → V1.1

Outil unique : Alembic + SQLAlchemy. Flyway et Liquibase ne figurent plus dans la recommandation. Une seule chaîne de révisions, exécutée en CI sur une base restaurée de production, avec `alembic upgrade head` puis `alembic downgrade -1` vérifiés à chaque livraison. La révision V1.1 est **non destructive** : aucune colonne supprimée, aucun type réduit, aucune donnée perdue. Seule exception assumée : la vue matérialisée `daily_sales_facts` est supprimée puis reconstruite comme table, ce qui est sans risque puisqu'elle ne porte aucune vérité.

Étape	Opération	Détail et précaution
1	Nouveaux types énumérés	CREATE TYPE pour return_refund_status, forecast_type, insight_type, insight_severity, insight_status. Ajout de 'initial' à la liste de ref_type (un CHECK, pas un ENUM : DROP puis ADD CONSTRAINT).
2	Colonnes additives	products.default_tax_rate, sale_items.tax_rate, sale_returns.refunded_total/refund_status, sale_return_items.tax_refund/unit_cost, payments.sale_return_id, inventory_movements.reversal_of_movement_id, product_stock.avg_cost. Toutes avec NOT NULL DEFAULT : en PostgreSQL 11+ l'ajout ne réécrit pas la table.
3	Rétro-remplissage des coûts	sale_return_items.unit_cost ← sale_items.unit_cost par jointure. inventory_movements.unit_cost NULL ← product_prices.cost_price valide à occurred_at, sinon 0 avec trace dans audit_logs. **À exécuter avant de passer la colonne en NOT NULL.**
4	Colonnes générées	sale_items.net_amount/tax_amount ajoutées ; line_total et sales.total **redéfinies** : DROP COLUMN puis ADD COLUMN … GENERATED dans la même migration. Une colonne générée ne peut pas être modifiée en place. L'ordre compte : les taux valant 0 après remplissage, les valeurs recalculées sont identiques aux anciennes.
5	Alignement de tax_total	UPDATE sales SET tax_total = 0 — les ventes historiques ont un taux nul, donc total reste inchangé. Contrôle obligatoire : SELECT count(*) FROM sales WHERE total <> subtotal - discount_total + tax_total doit renvoyer 0.
6	CMP initial	SELECT rebuild_product_stock(id) FROM companies : rejoue le journal et calcule le CMP historique de chaque article. C'est l'étape la plus longue — la planifier hors production ouverte, entreprise par entreprise.
7	Nouvelles tables	sale_return_refunds, forecasts, insights, ml_feature_snapshots, avec leurs FK composites et index.
8	daily_sales_facts	DROP MATERIALIZED VIEW → CREATE TABLE → reconstruction complète sur l'historique par la requête de 6.1. Aucune reprise de données de l'ancienne MV : ses chiffres étaient faux par duplication.
9	Contraintes et index	inv_reversal_uq, remplacement de l'index d'idempotence par sa version partielle (DROP INDEX + CREATE UNIQUE INDEX … WHERE), CHECK de direction sur payments, plafonds de remboursement. Créer les index en CONCURRENTLY hors transaction pour les grosses tables.
10	Fonctions et triggers	cmp_next_avg, inv_check_reversal, inv_apply_movement (remplace la version V1), rebuild_product_stock, triggers de retour et de remboursement. Remplacement des RULES DO INSTEAD NOTHING par les triggers d'immuabilité.
11	RLS	Bloc DO de 8.11 : politiques sur audit_logs et sur les nouvelles tables. Vérifier ensuite pg_class.relrowsecurity et relforcerowsecurity sur les 28 tables tenant-scoped.
12	Contrôle post-migration	Rejeu de la suite de tests de la section 10 sur la base migrée, avant réouverture du service.

```
# alembic/versions/0002_v11_corrective.py — extraits representatifs
revision = "0002_v11_corrective"; down_revision = "0001_v1_initial"

def upgrade():
    op.add_column("products", sa.Column("default_tax_rate",
        sa.Numeric(6, 4), nullable=False, server_default="0"))
    op.add_column("inventory_movements", sa.Column(
        "reversal_of_movement_id", postgresql.UUID(as_uuid=True), nullable=True))
    op.create_foreign_key("inv_reversal_fk", "inventory_movements",
        "inventory_movements",
        ["company_id", "reversal_of_movement_id"], ["company_id", "id"],
        ondelete="RESTRICT")
    op.create_unique_constraint("inv_reversal_uq", "inventory_movements",
        ["company_id", "reversal_of_movement_id"])
    op.drop_index("inv_ref_idempotence", table_name="inventory_movements")
    op.execute("""CREATE UNIQUE INDEX inv_ref_idempotence
        ON inventory_movements (company_id, ref_type, ref_id)
        WHERE ref_id IS NOT NULL AND reversal_of_movement_id IS NULL""")
    op.execute("SELECT rebuild_product_stock(id) FROM companies")

def downgrade():
    # Reversible sur le schema. Les valeurs de CMP calculees ne sont pas
    # restaurables : documente comme perte acceptable d'une projection.
    ...
```

10. Tests d'intégrité indispensables

Chaque test est un *pytest* qui ouvre une transaction, pose `SET LOCAL app.company_id` et attend soit un résultat, soit une exception PostgreSQL précise. Un test qui passerait parce que l'API a vérifié quelque chose ne vaut rien : tous ces tests attaquent la base directement.

#	Test	Scénario	Attendu
T1	Isolation en lecture	Contexte tenant A, SELECT count(*) sur customers, sales, product_stock, payments, audit_logs, forecasts, insights, daily_sales_facts contenant des données de B	0 ligne sur les 8 tables
T2	Isolation en écriture	Contexte A, INSERT avec company_id = B	WITH CHECK violé
T3	Contexte absent	Aucun SET LOCAL, SELECT sur sales	0 ligne, jamais d'erreur ni de fuite
T4	Fuite par le pool	Transaction A puis réutilisation de la même connexion pour B sans nouveau SET LOCAL	0 ligne : SET LOCAL meurt au COMMIT
T5	FK croisée	sale_items de A référençant un produit de B ; idem sale_return_refunds → paiement de B	Violation de FK composite
T6	Stock négatif — vente	Stock 5, vente de 6	Exception, transaction annulée, stock resté à 5
T7	Stock négatif — autres sorties	adjustment −10, loss −10, supplier_return_out −10 sur stock 5	Exception dans les 3 cas
T8	Article non suivi	Mouvement sur un produit is_stock_tracked = false	Exception ; aucune ligne de product_stock créée
T9	CMP — entrée	100 u à 10, puis 100 u à 20	avg_cost = 15.0000, stock_value = 3000.00
T10	CMP — sortie	Après T9, sortie de 50	unit_cost du mouvement = 15 ; avg_cost toujours 15 ; stock_value = 2250.00
T11	CMP — stock nul	Sortie de la totalité	qty_on_hand = 0, avg_cost conservé, aucune division par zéro
T12	CMP — reconstruction	rebuild_product_stock après 200 mouvements aléatoires	Écart nul sur qty et sur avg : trigger et recalcul donnent le même résultat
T13	Immuabilité du journal	UPDATE et DELETE sur inventory_movements	Exception restrict_violation dans les deux cas
T14	Annulation de vente	Vente confirmée puis annulée	Un mouvement inverse par ligne, reversal_of_movement_id renseigné, stock revenu à l'identique
T15	Idempotence d'annulation	Deuxième annulation de la même vente	Violation de inv_reversal_uq ; aucun mouvement supplémentaire
T16	Annulation d'annulation	Inverse d'un mouvement déjà inverse	Exception du trigger de validation
T17	Idempotence de sortie	Deux mouvements pour le même sale_item_id	Violation de l'index partiel d'idempotence

#	Test	Scénario	Attendu
T18	Retours multiples	Ligne de 10 retournée 3, 4 puis 4	Les deux premiers passent, le troisième lève ; cumul plafonné à 10
T19	Coût de retour	Vente au CMP 15, CMP passé à 20, puis retour	Entrée à 15 (coût de sortie), pas à 20 ; cogs_returned compense exactement cogs_gross
T20	Remboursement	Retour de 1 000 remboursé 600 puis 500	Le premier passe, le second lève ; refund_status = partial
T21	Sens du remboursement	payment inflow avec sale_return_id	Violation du CHECK payments_return_outflow
T22	Traçabilité du remboursement	Requêtes croisées retour → paiement et paiement → retour	Réponse unique et non ambiguë dans les deux sens
T23	Taxes figées	Vente à 18 %, puis products.default_tax_rate passé à 5 %	sale_items.tax_amount et sales.total inchangés
T24	Égalité comptable	Balayage de toutes les ventes	subtotal − discount_total + tax_total = total partout
T25	Paiement partiel	Dette 1 000, paiement 400	amount_paid = 400, balance = 600, status = partially_paid
T26	Paiement multi-dettes	Paiement 1 000 réparti 400 / 400 / 300	La troisième allocation lève : sur-affectation
T27	Dette soldée	Solde atteint 0	status = paid, closed_at renseigné
T28	Plafond de crédit	Nouvelle dette dépassant credit_limit	Exception du trigger
T29	Agrégation — duplication	Une ligne de vente avec 3 retours partiels, puis reconstruction	qty_sold, gross_revenue, discount_total, cogs_gross et sale_count identiques au cas sans retour ; **c'est le test qui échouait en V1**
T30	Agrégation — idempotence	Reconstruction de la même plage deux fois	Résultat identique ; aucune ligne dupliquée
T31	Agrégation — annulation	Vente annulée après construction du fait	Ligne de fait purgée ; aucun fait fantôme
T32	Cohérence net_revenue	Comparaison au calcul direct depuis le transactionnel	Égalité à 0,01 près sur 90 jours
T33	Idempotence des résultats ML	Rejeu du pipeline sur le même as_of	forecasts : mise à jour, pas de doublon ; insights : même dedupe_key mise à jour
T34	Insight écarté	Insight dismissed puis réémission de la même dedupe_key	Insertion acceptée : l'index partiel ne bloque pas
T35	Pas d'inférence en ligne	Trace SQL d'un appel aux écrans de prévision	Aucune lecture de ml_feature_snapshots, aucun calcul : seulement forecasts / insights par index
T36	Rôle applicatif	Inspection de pg_roles	rolbypassrls = false, rolsuper = false ; aucun DELETE physique accordé
T37	Couverture RLS	pg_class croisé à la liste des tables tenant-scoped	28/28 avec relrowsecurity et relforcerowsecurity

#	Test	Scénario	Attendu
T38	Concurrence sur le stock	Deux transactions vendant simultanément le dernier article	Une réussit, l'autre lève ; jamais de stock négatif ni de CMP faux
T39	Atomicité de la vente	ROLLBACK injecté après les mouvements	Ni vente, ni mouvement, ni événement ; stock intact
T40	Outbox	Crash simulé après COMMIT, avant publication	Événement toujours présent avec published_at IS NULL ; rejouable

11. Matrice V1 / V1.1 / futur

Évolution	Statut	Ce qui existe en V1.1	Ce qu'il resterait à faire
Snapshot des taxes	**Implémenté**	tax_rate / tax_amount figés, tax_total, égalité comptable générée	—
Valorisation CMP	**Implémenté**	avg_cost, stock_value, coût de sortie estampillé, reconstruction	—
Stock négatif interdit	**Implémenté**	CHECK + exception trigger, tous types de sortie	—
Mouvements inverses tracés	**Implémenté**	reversal_of_movement_id, unicité, validation	—
Remboursement traçable	**Implémenté**	sale_return_refunds, payments.sale_return_id, plafonds	—
Résultats ML persistés	**Implémenté**	forecasts, insights, idempotence, RLS	Registre de modèles, backtest automatisé (hors base)
Analytique isolée	**Implémenté**	daily_sales_facts table + RLS, retours pré-agrégés	Partitionnement mensuel si le volume l'exige
RLS exhaustive	**Implémenté**	28 tables, audit_logs inclus, rôles sans BYPASSRLS	—
Migrations Alembic	**Implémenté**	Chaîne unique V1 → V1.1, non destructive	—
Variantes produit	Préparé	product_id comme seule référence stable ; stock événementiel et prix historisés, donc reventilables	Table product_variants + variant_id nullable sur 4 tables + reconstruction de product_stock
Multi-company / memberships	Préparé	Unicité e-mail **par tenant**, company_id jamais dérivé de l'utilisateur dans les FK	Table company_memberships, choix de tenant après authentification, revue du flux JWT
Multi-entrepôts	Préparé	Stock entièrement événementiel ; product_stock jetable	warehouses + location_id sur les mouvements + clé de projection élargie
Méthode FIFO	Préparé	Journal complet quantité / coût / date : recalcul historique possible	Table de couches de coût, ou recalcul à la demande
Promotions	Préparé	Remise stockée séparément du prix, donc distinguable d'une baisse tarifaire durable	promotions + rattachement aux lignes
Multi-devises	**Hors V1**	companies.currency unique ; tous les montants dans cette devise	currency + taux sur chaque flux, revue de toutes les agrégations
Avoirs clients	**Hors V1**	Statut tranché en 4.6 : aucune table, aucun solde créditeur	credit_notes + solde client + imputation sur ventes futures
Comptabilité générale	**Hors V1**	—	Plan comptable, écritures à double entrée, journaux, exercices

Évolution	Statut	Ce qui existe en V1.1	Ce qu'il resterait à faire
Moteur fiscal	**Hors V1**	Un taux par ligne, figé. Pas de règles, pas de territorialité	Table de règles fiscales, exonérations, déclarations
CRM	**Hors V1**	Fiche client minimale	Interactions, segments, campagnes
Achats avancés	**Hors V1**	Entrées de stock + charge ou dette fournisseur	purchase_orders, réceptions partielles, rapprochement de factures
Event sourcing complet	**Hors V1**	Outbox business_events uniquement, l'état reste la vérité	Non souhaitable pour CoGes : sur-ingénierie assumée comme écartée

12. Critère final

> *« Le modèle V1.1 est-il maintenant suffisamment cohérent pour commencer l'implémentation backend FastAPI + PostgreSQL + Alembic sans devoir refaire le schéma sur les mêmes sujets ? »*

Oui — sur les dix sujets corrigés, aucun ne devrait rouvrir le schéma.

Les quatre décisions ouvertes sont verrouillées et portées par des structures, pas par des intentions. Les dix défauts recensés en section 1 sont chacun corrigés par une contrainte, un index ou une table — pas par une convention applicative. Les propriétés critiques (isolation, non-négativité du stock, idempotence de l'annulation, plafond de remboursement, égalité comptable) sont désormais **vérifiables par un test SQL qui attaque la base directement**, ce qui est le seul critère fiable de non-régression.

12.1 Trois points résiduels — non masqués

Aucun n'est bloquant, aucun n'exige de refaire le schéma. Les trois sont des **risques de processus**, à traiter dans l'implémentation.

Point	Nature exacte du risque	Correction minimale avant développement
1. SET LOCAL et pool de connexions	C'est le seul endroit où une erreur d'implémentation peut encore produire une fuite entre tenants : un SET au lieu d'un SET LOCAL, ou un SET LOCAL posé hors transaction, survivrait au retour de la connexion au pool. La base ne peut pas s'en protéger seule.	Poser le contexte dans une **dépendance FastAPI unique**, jamais ailleurs ; interdire SET par revue de code ; T3, T4 et T37 en test de non-régression obligatoire à chaque livraison.
2. CMP et antidatage	Le CMP dépend de l'ordre des mouvements. Un mouvement inséré avec un occurred_at antérieur au dernier mouvement existant rend le CMP courant différent du CMP recalculé. Le schéma ne l'interdit pas ; c'est inhérent à toute valorisation en coût moyen.	Deux options, à trancher avant développement : refuser l'antidatage au-delà d'une fenêtre (par exemple 48 h) par un CHECK sur occurred_at, ou l'autoriser et accepter l'écart corrigé chaque nuit par rebuild_product_stock avec trace dans audit_logs. **Recommandation : la première**, plus simple à expliquer au gérant.
3. Fenêtre de reconstruction analytique	Les retours étant rattachés au jour de la vente, l'exactitude de daily_sales_facts dépend d'un *job* qui déduit correctement les jours à réécrire depuis business_events. Un événement manquant produit un agrégat silencieusement faux — une contrainte SQL ne peut pas détecter cela.	Écrire l'événement **dans la même transaction** que chaque mutation de vente, de retour et d'annulation, sans exception ; ajouter un contrôle de réconciliation hebdomadaire (T32 sur 90 jours) et une alerte si l'écart dépasse 0,01.

Périmètre fonctionnel : inchangé. Aucune fonctionnalité n'a été ajoutée ni retirée silencieusement. Les taxes, seule zone qui pouvait être interprétée de deux façons, sont explicitement **dans** le périmètre sous une forme minimale ; les avoirs clients, la comptabilité générale, le multi-devises et les variantes sont explicitement **hors** périmètre, avec leur point d'extension documenté.

Ordre de développement recommandé : (1) socle tenant + RLS + dépendance de contexte, avec T1–T5 et T36–T37 avant toute autre ligne de code ; (2) référentiels et prix ; (3) stock et CMP, avec T6–T13 et T38 ; (4) ventes, confirmation, annulation, avec T14–T17, T23–T24, T39 ; (5) retours et remboursements, T18–T22 ; (6)

dettes et paiements, T25–T28 ; (7) outbox et couche analytique, T29–T32, T40 ; (8) forecasts et insights, T33–T35. Le ML arrive en dernier, et seulement en lecture.